

Operating System Concepts

A. Silberschatz, P. B. Galvin, and G. Gagne, 10th edition

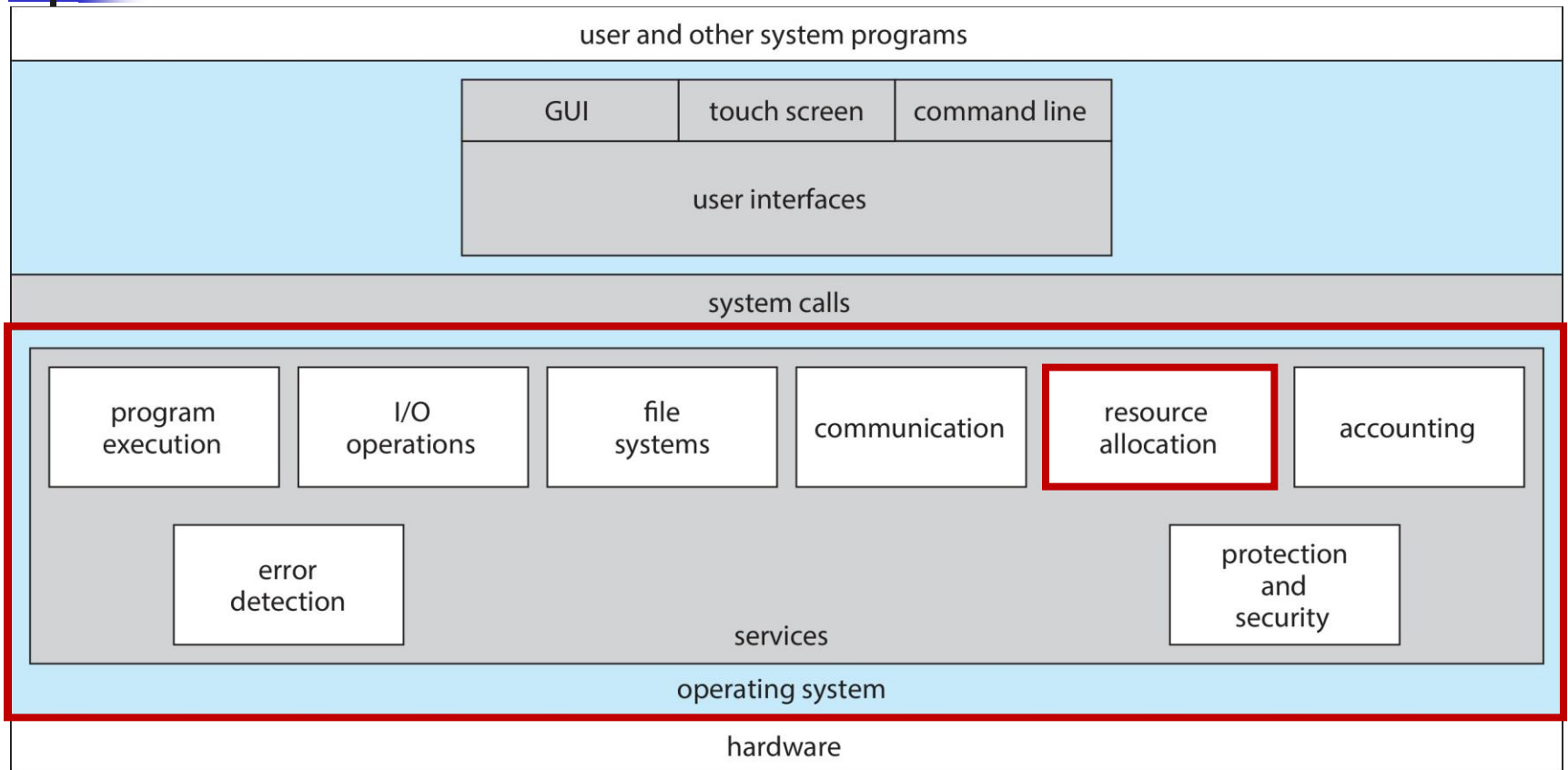
Chapter 9: Main Memory

Mei-Ling Chiang, Professor

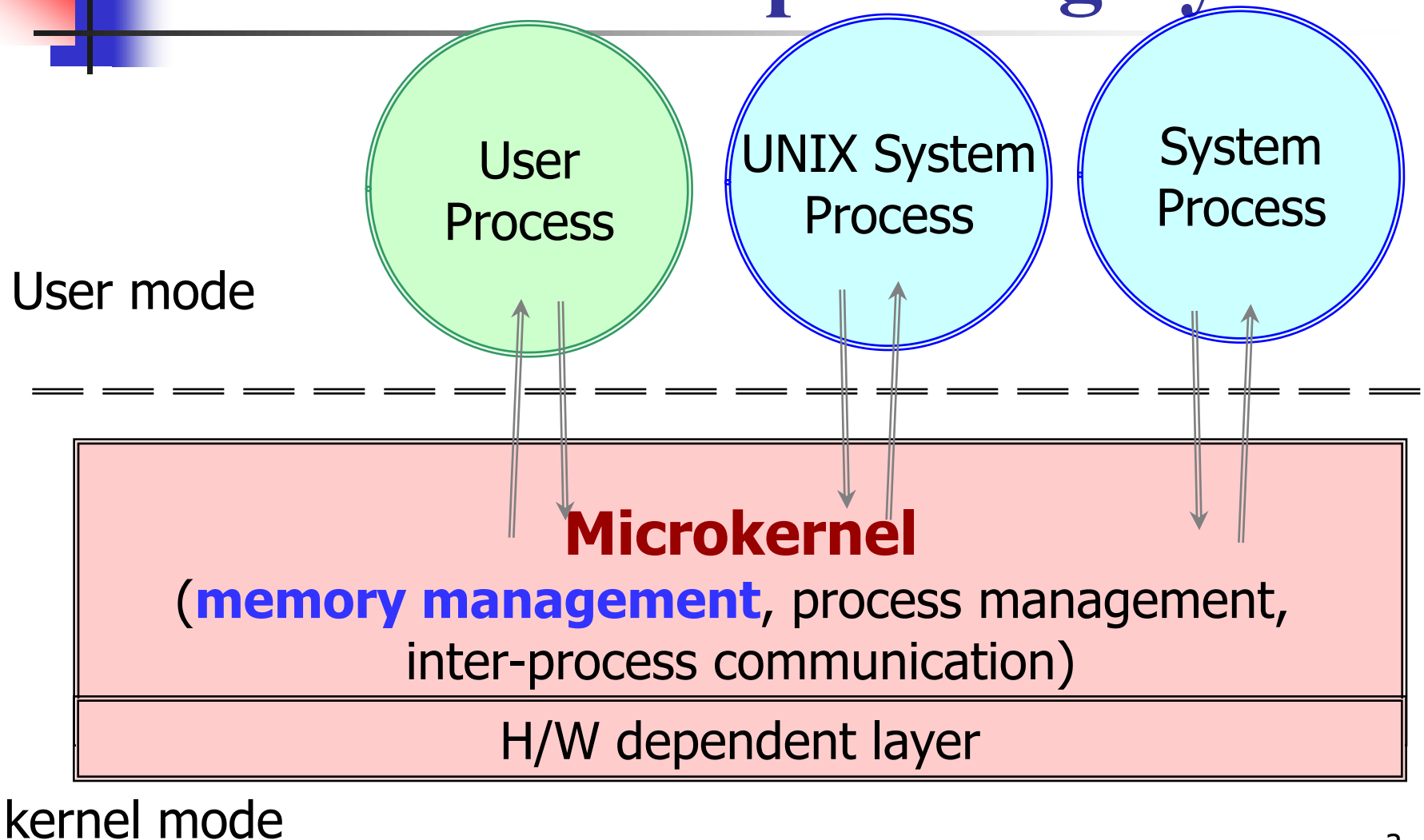
**Dept. of Information Management
National Chi-Nan University, Taiwan, ROC**

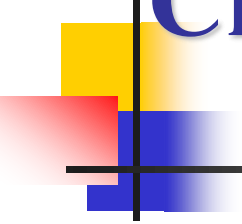


A View of OS Services (Fig. 2.1)



Microkernel Operating System

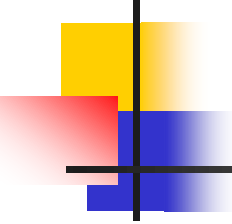




Chapter 9: Main Memory

Outline

- 9.1 Background 
- 9.2 Contiguous Memory Allocation 
- 9.3 Paging 
- 9.4 Structure of the Page Table 
- 9.5 Swapping 
- 9.6 Example: The Intel 32 and 64 bit Architectures 
- 9.7 Example: ARMv8 Architecture
- 9.8 Summary



Memory Management

- To improve both CPU utilization and response time to users, several processes must be kept in memory must *share* memory) .
- Several memory management schemes
 - vary from a primitive bare-machine approach to paging strategy
 - selection of a memory-management method depends on the *hardware* design
 - most algorithms require hardware support

Concurrent Execution

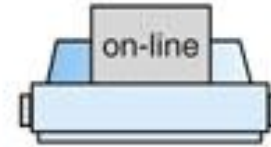
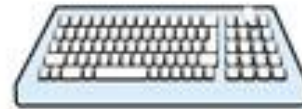
Application programs

disks

mouse

keyboard

printer



CPU

disk
controller

USB controller

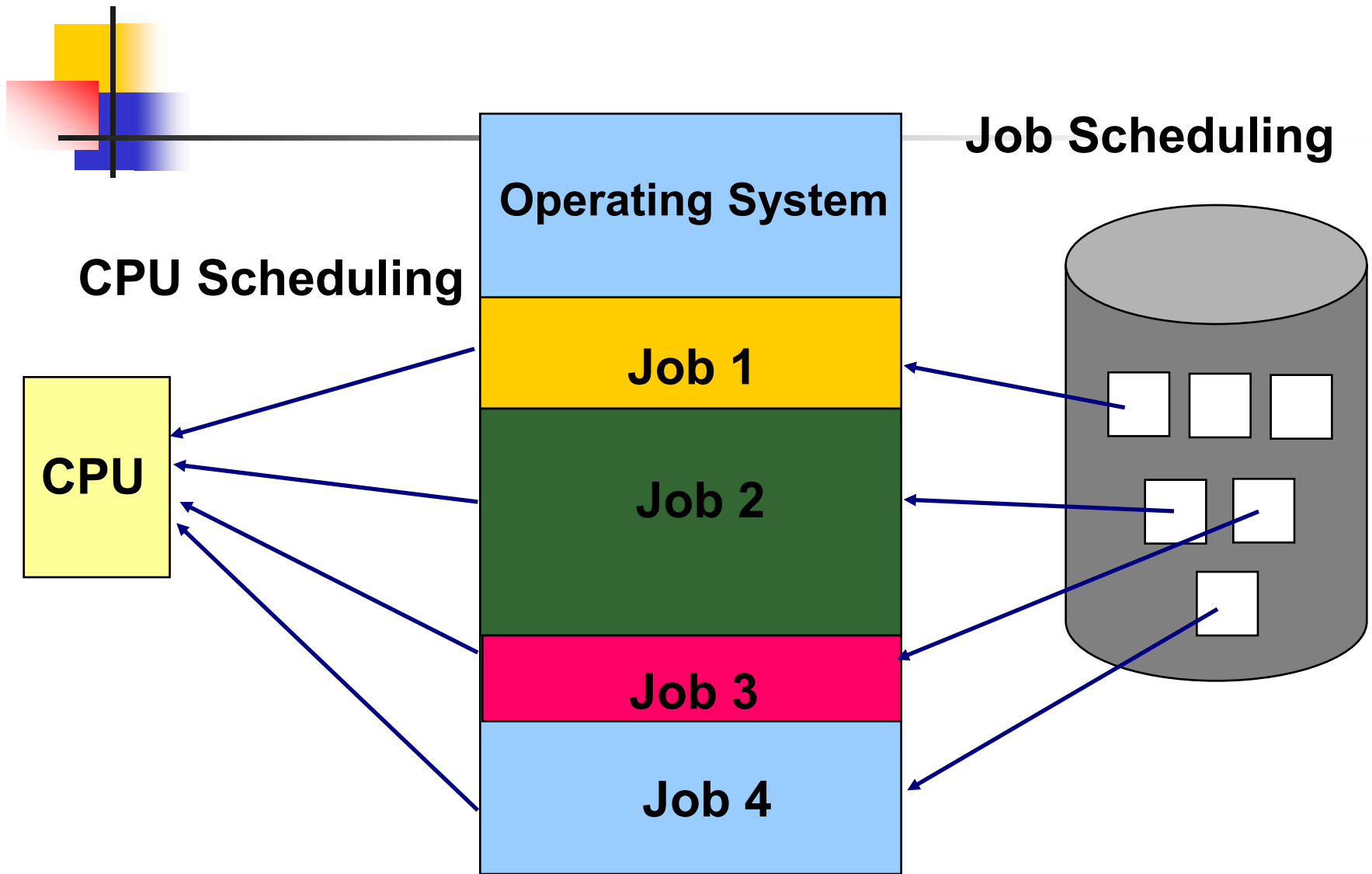
graphics
adapter

OS

Process
#1

Process
#2

RAM

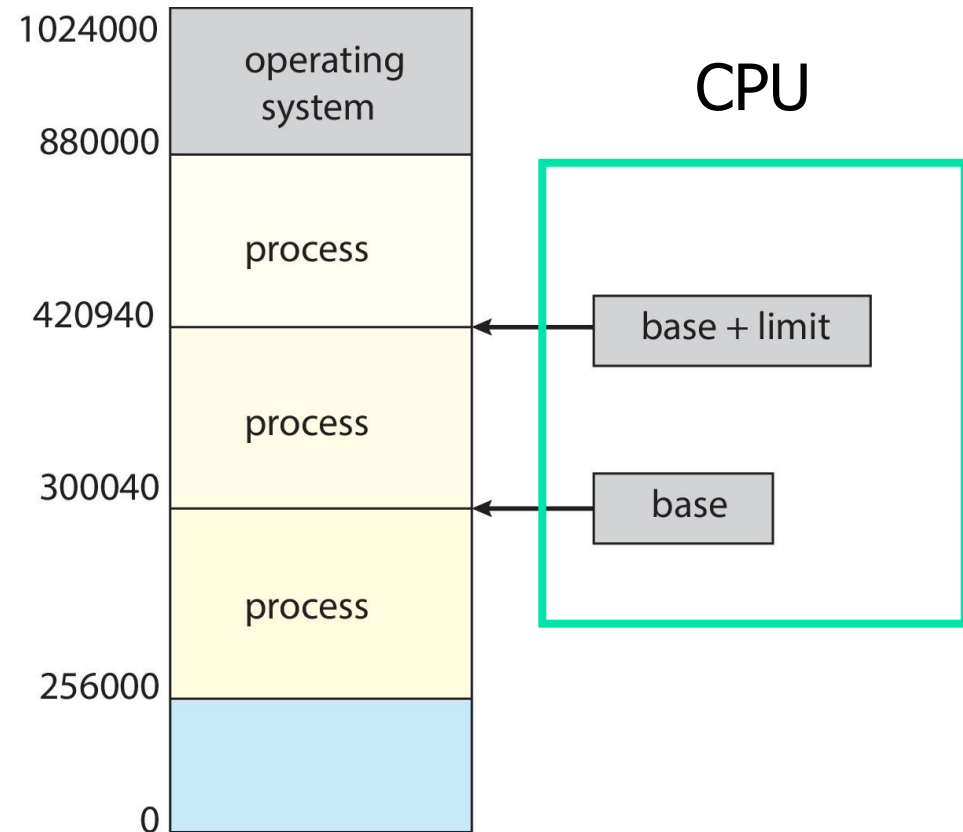


Memory Management

- **Memory-management information in Process Control Block**

- **base and limit registers**

Hardware support

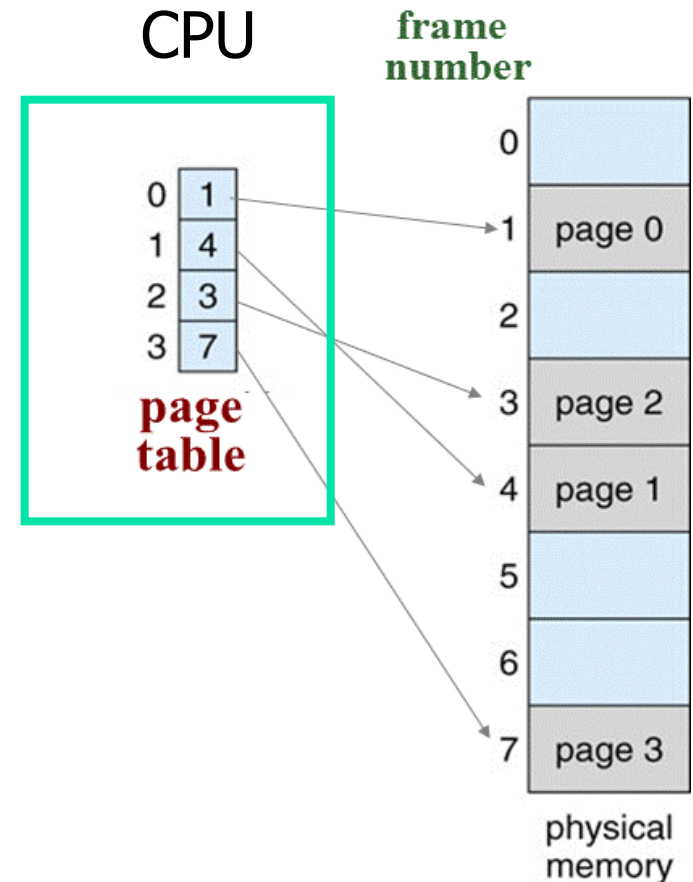
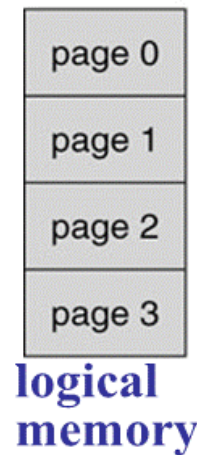


Memory Management

- **Memory-management information in Process Control Block**

- **page table**

Paging hardware



Memory Management

- **Memory-management information in Process Control Block**

- **page table**

Paging hardware

page 0
page 1
page 2
page 3

logical
memory

CPU

MMU

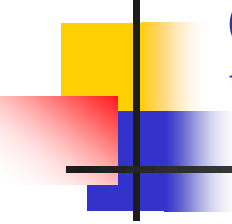
frame
number

0	1
1	4
2	3
3	7

page
table

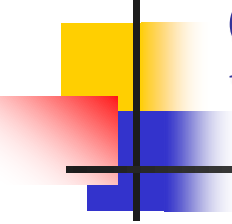
0	
1	page 0
2	
3	page 2
4	page 1
5	
6	
7	page 3

physical
memory



9.1 Background

- **9.1.1 Basic Hardware**
- **9.1.2 Address Binding**
- **9.1.3 Logical vs. Physical Address Space**
- **9.1.4 Dynamic Loading**
- **9.1.5 Dynamic Linking and Shared Libraries**



9.1.1 Basic Hardware

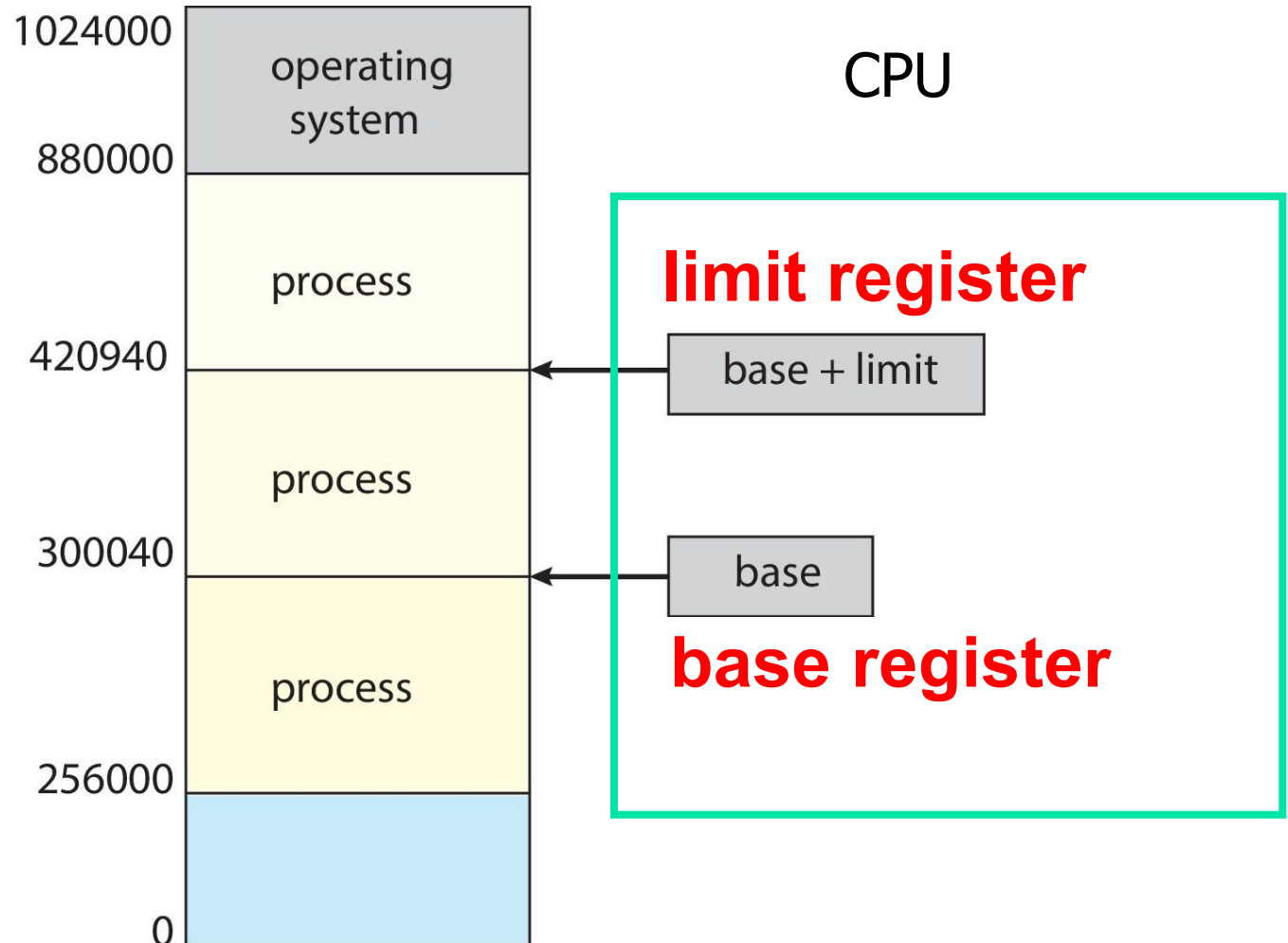
- We must ensure correct operation to protect OS from access by user processes, and to protect user processes from one another.
- **Memory protection** must be provided by H/W.
- Need to make sure that each process has a separate memory space.

Hardware-assisted Memory Protection

■ Example:

- In order to have memory protection, add two registers that determine the range of legal addresses a process may access:
 - base register – holds the smallest legal physical memory address
 - limit register – contains the size of the range
- Memory outside the defined range is protected.
 - CPU hardware compares *every* address generated in user mode with the registers

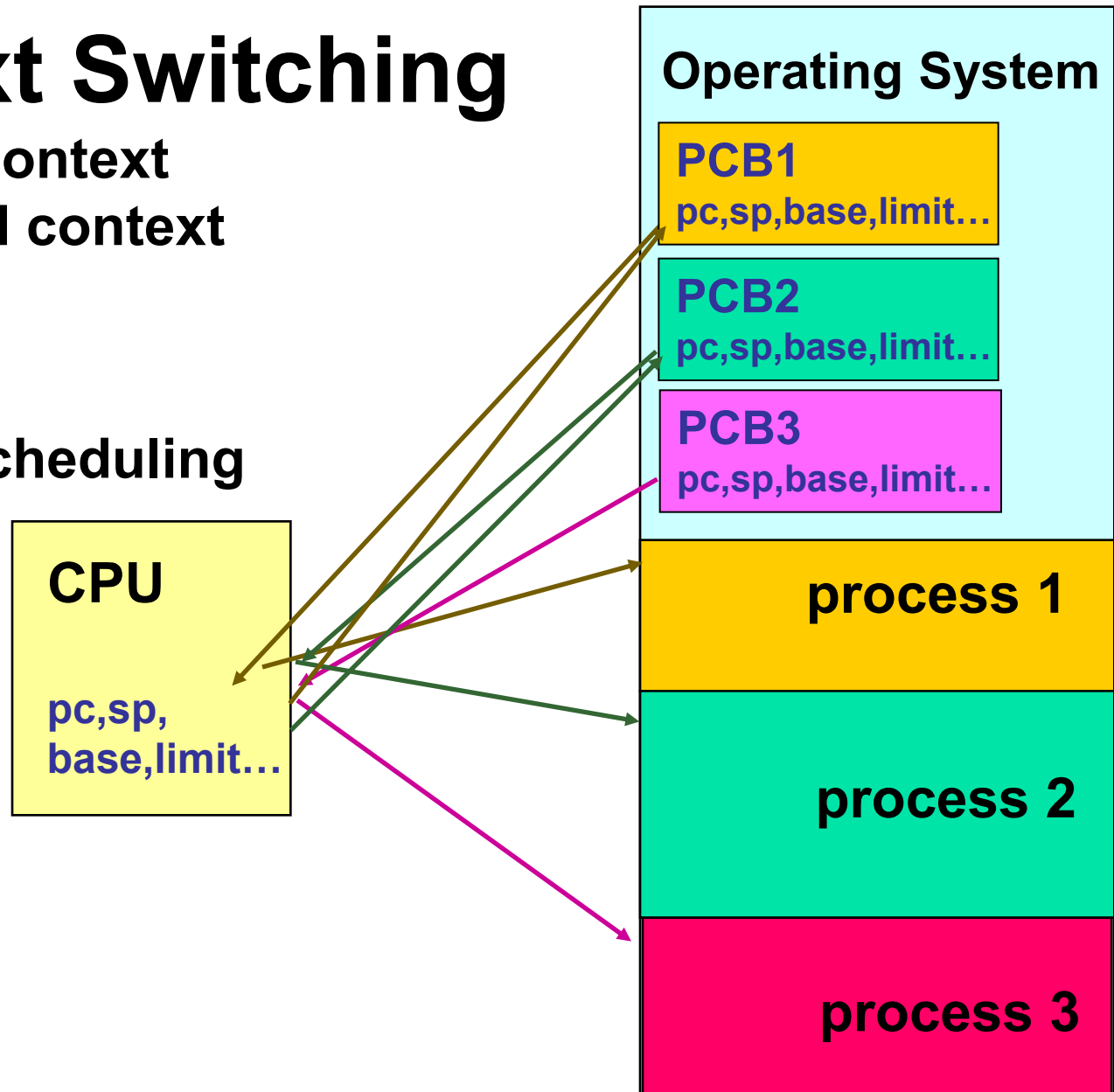
A base and a limit registers define a logical address space



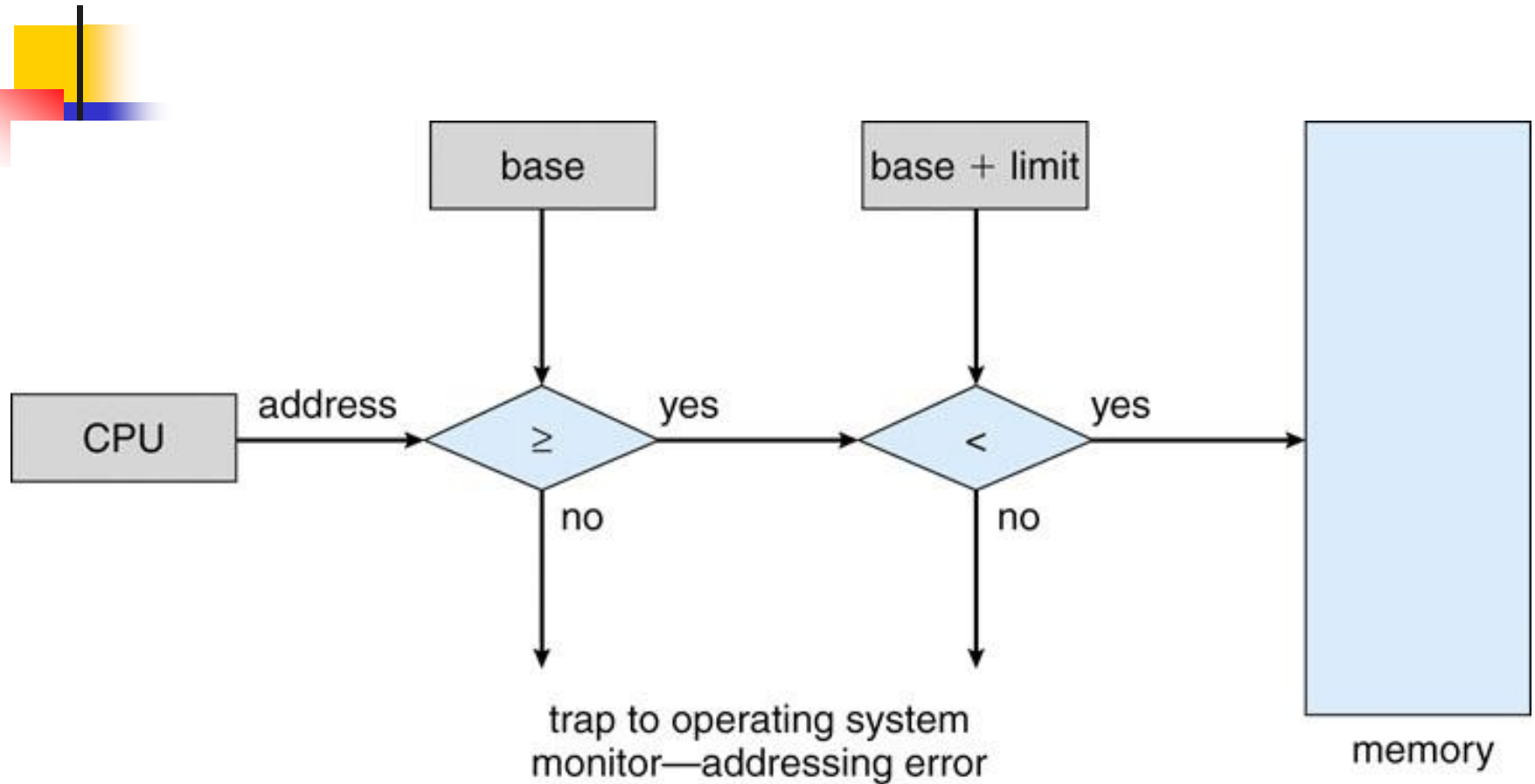
Context Switching

- Save context
- Reload context

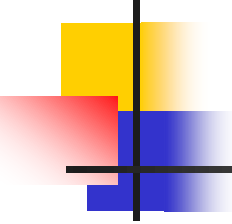
CPU Scheduling



Protection Hardware

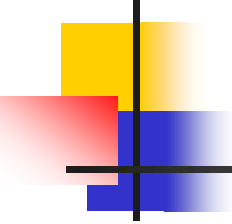


- Hardware address protection with base and limit registers.



Protection Hardware (Cont.)

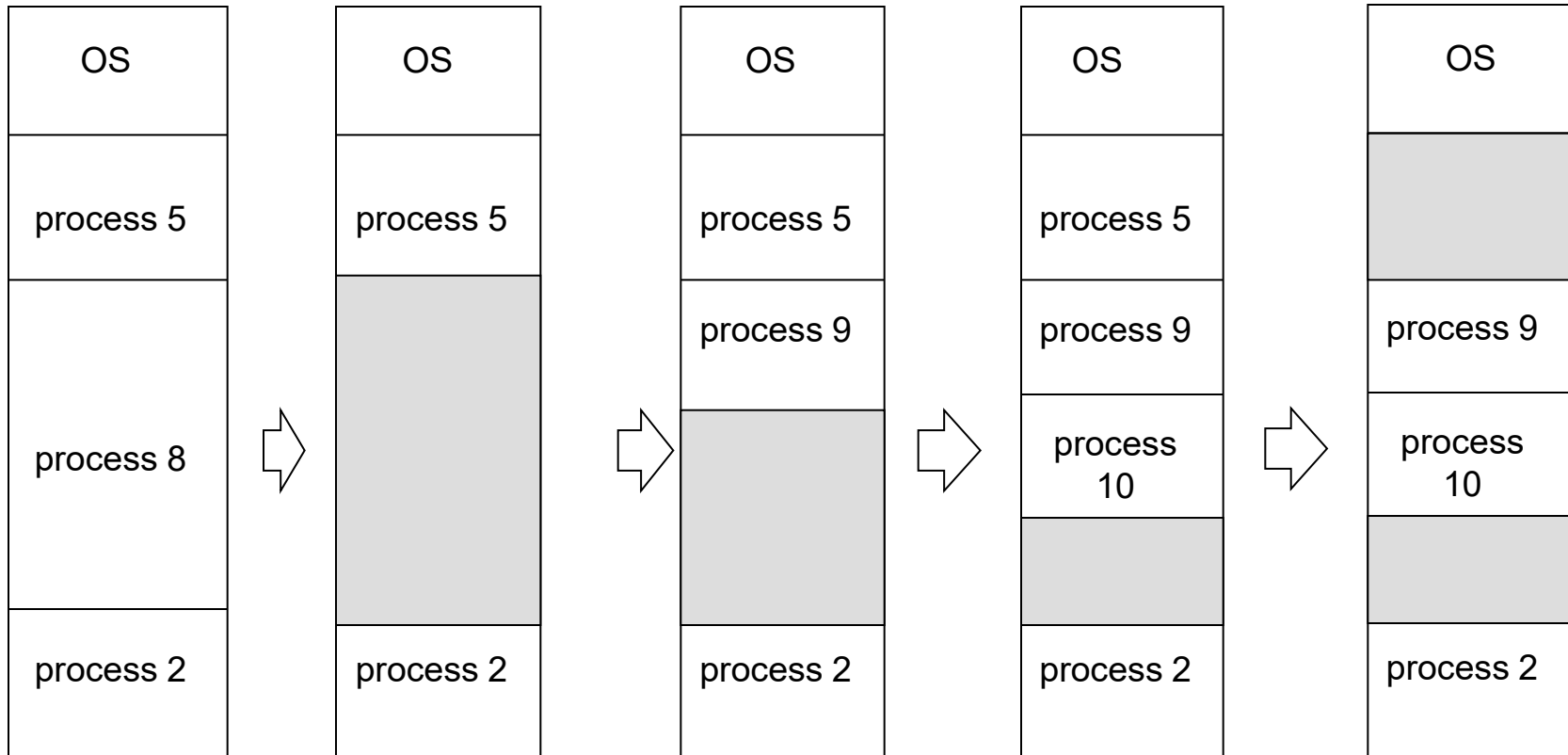
- The load instructions for the base and limit registers are privileged instructions.
 - So only the OS can load the base and limit registers.
- When executing in kernel mode, the OS has unrestricted access to both OS and user's memory.



9.1.2 Address Binding

- Program must be brought into memory and placed within a process for it to be executed.
- *Input queue* – collection of processes on the disk that are waiting to be brought into memory to run the program.
- Most systems allow a user process to reside in any part of the physical memory.

Example: Multiple-Partition Allocation

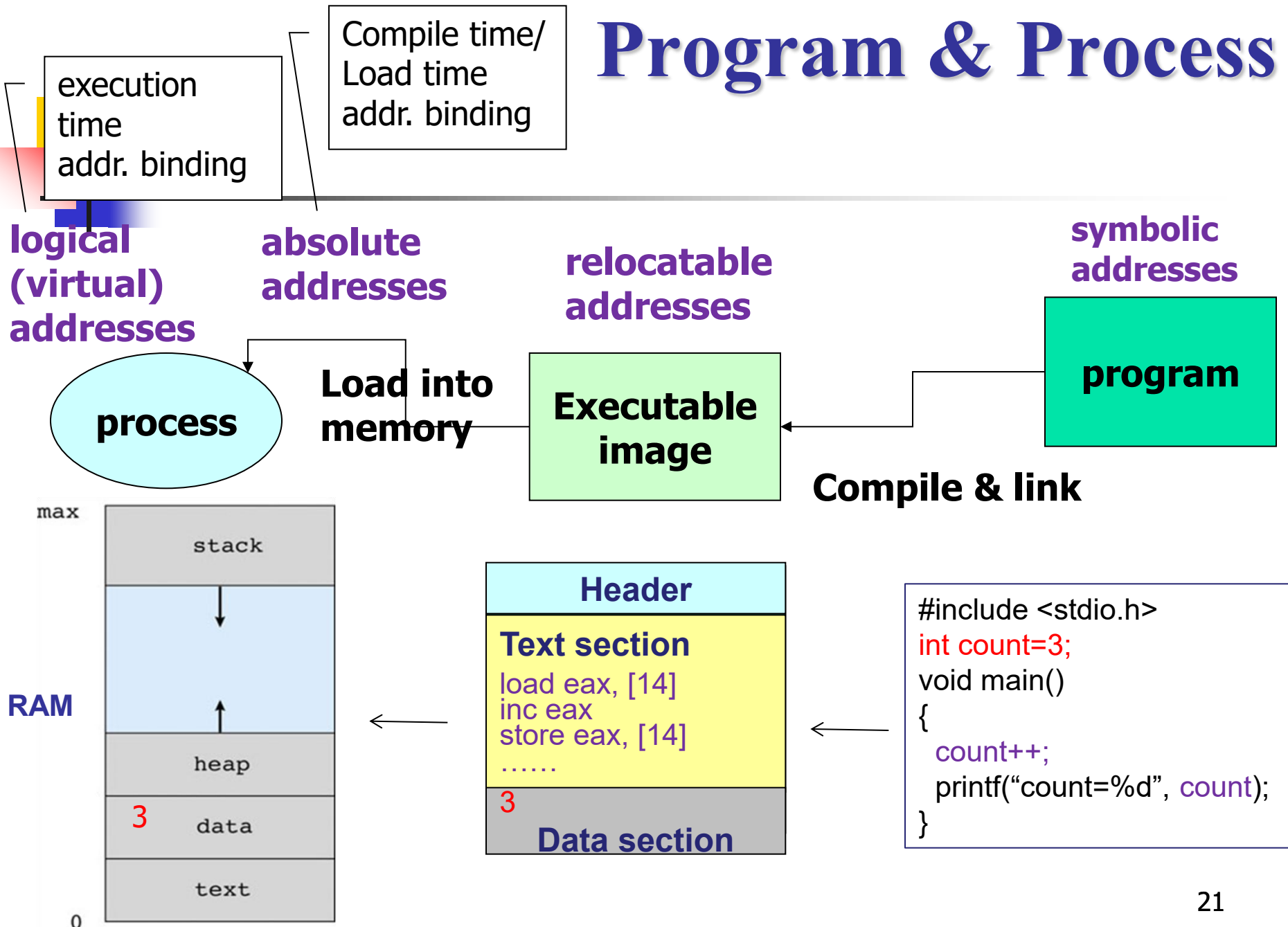




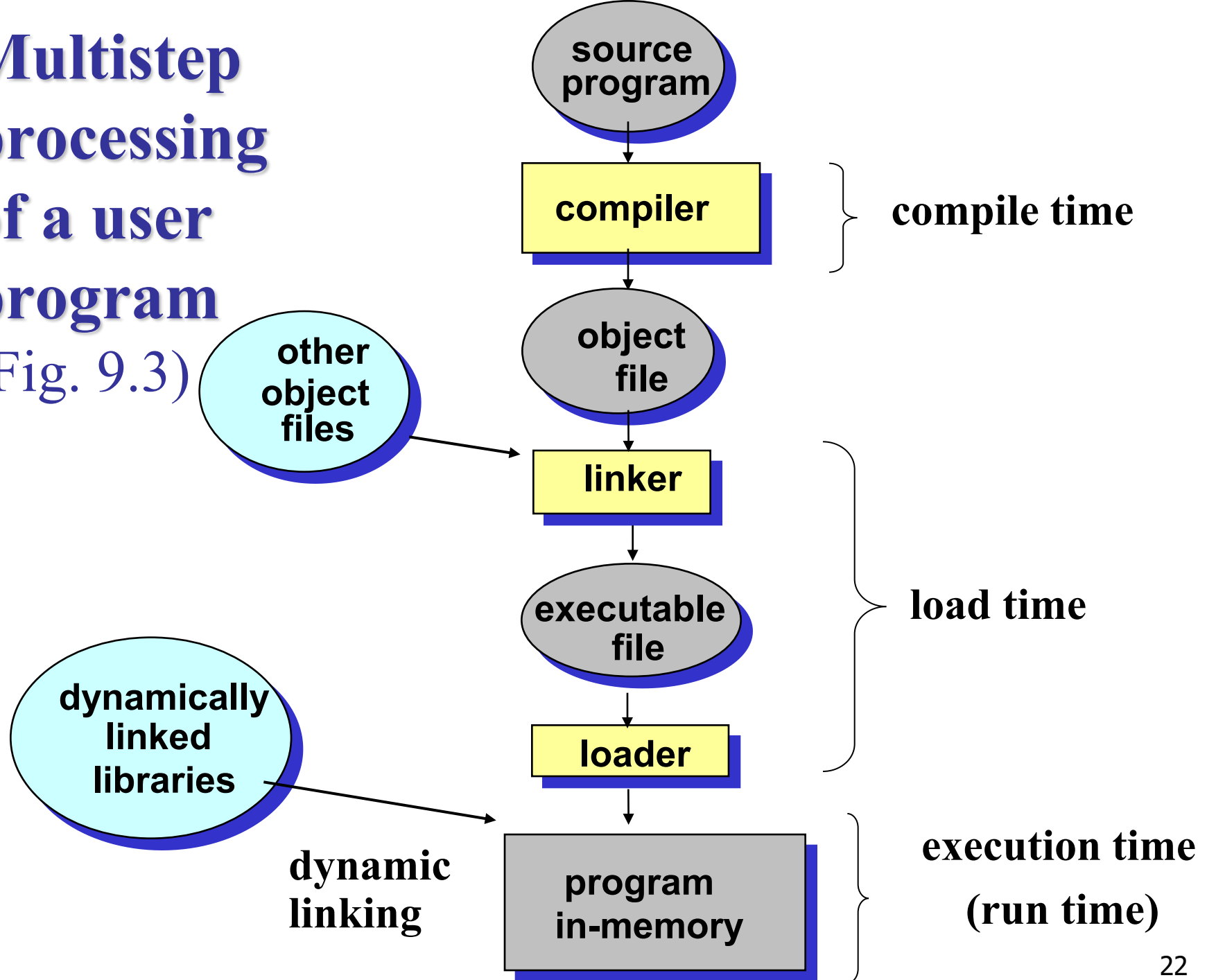
Address Binding (Cont.)

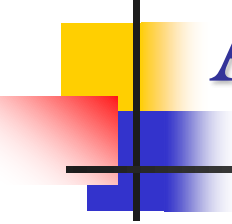
- User programs go through several steps before being executed. (Fig. 9.3)
- **Addresses** may be represented in different ways during these steps.
 - Addresses in the source program are generally **symbolic** (e.g., count)
 - A compiler will typically **bind** these symbolic addresses to **relocatable addresses** (e.g., “14” bytes from the beginning of this module)
 - The linkage editor or loader will in turn bind these relocatable addresses to **absolute addresses** (e.g., 74014)
- **Each binding is a mapping from one address space to another.**

Program & Process



Multistep processing of a user program (Fig. 9.3)





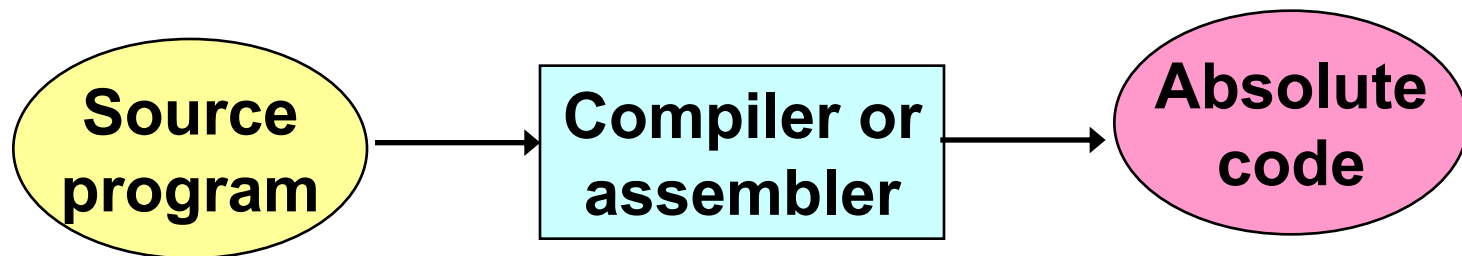
Address Binding (Cont.)

- **Classically, the binding of instructions and data to memory addresses can be done at any step along the way:**
 - **Compile time** 
 - **Load time** 
 - **Execution time** 



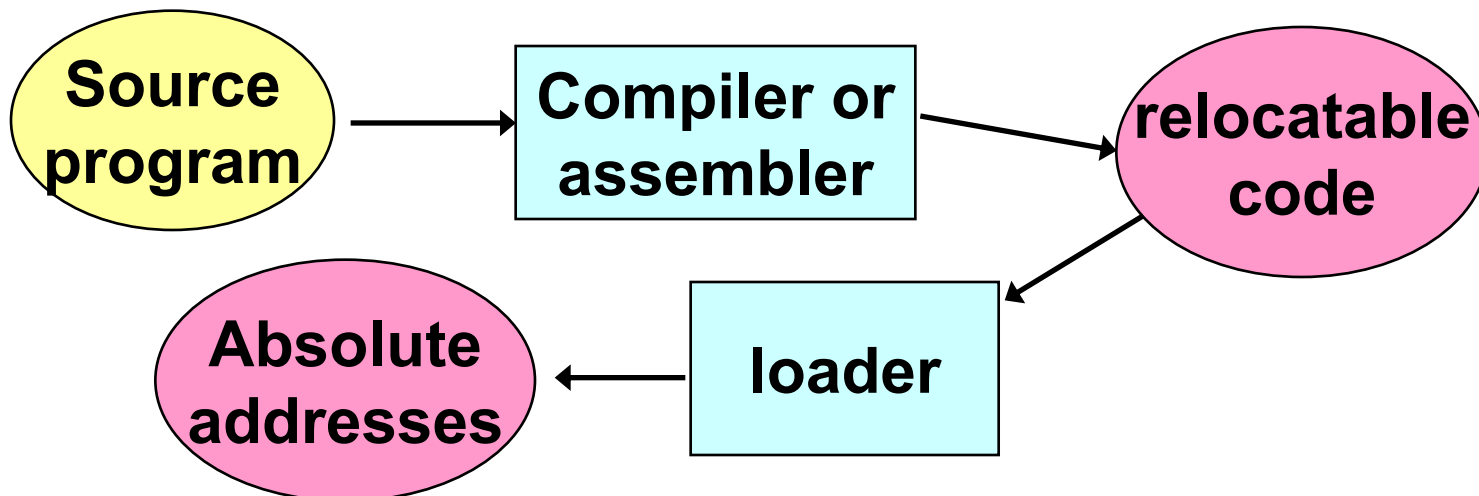
Compile-Time Address Binding

- If memory location is known at **compile time** where the process will reside, then **absolute code** can be generated
 - Must recompile code if starting location changes.
 - eg., the .COM-format programs in MS-DOS



Load-Time Address Binding

- If memory location is not known at compile time where the process will reside, then compiler must generate **relocatable code**
- Mapping from relocatable codes to absolute addresses at **load time**



Example: relocation 重新定址 改變位置

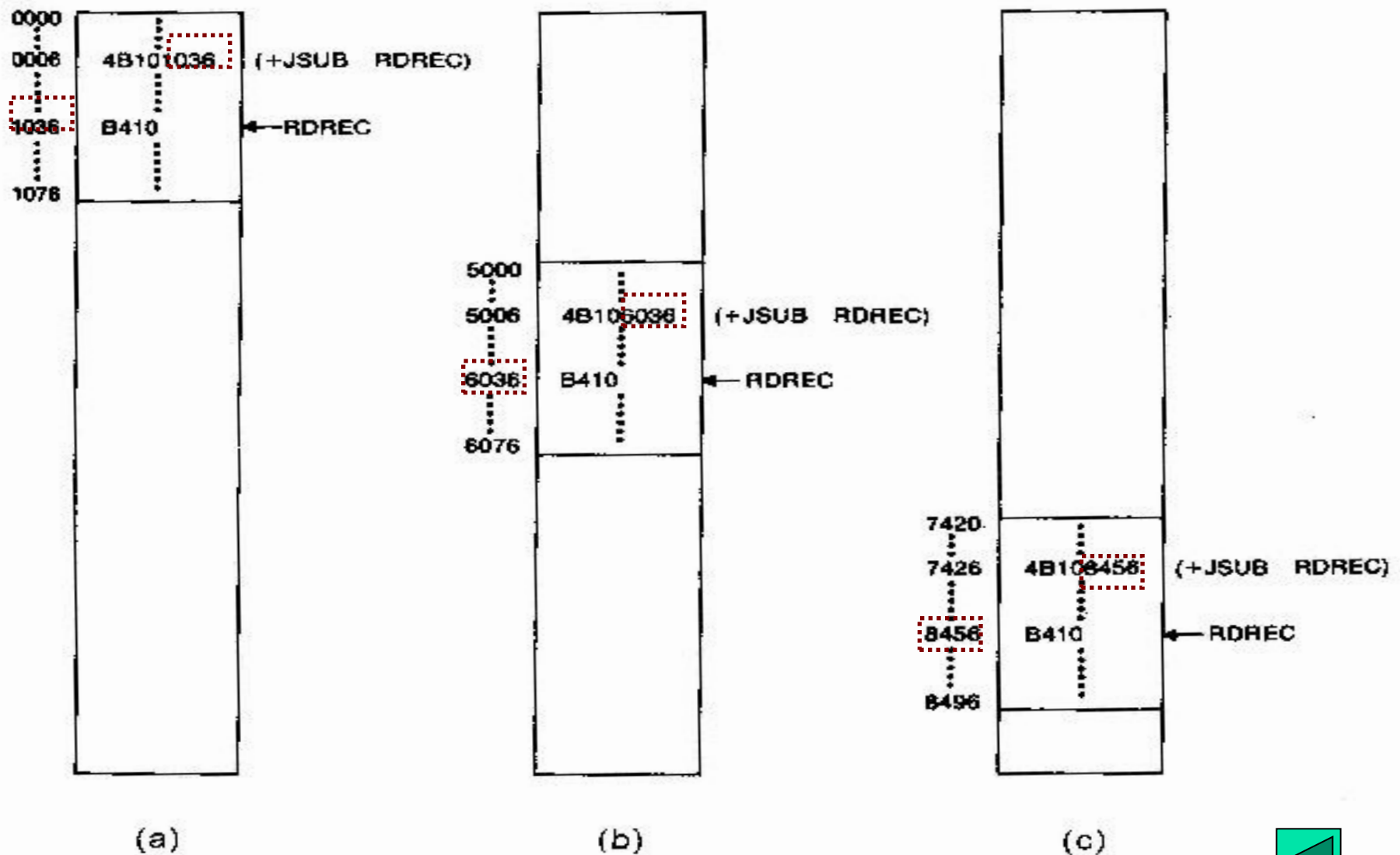
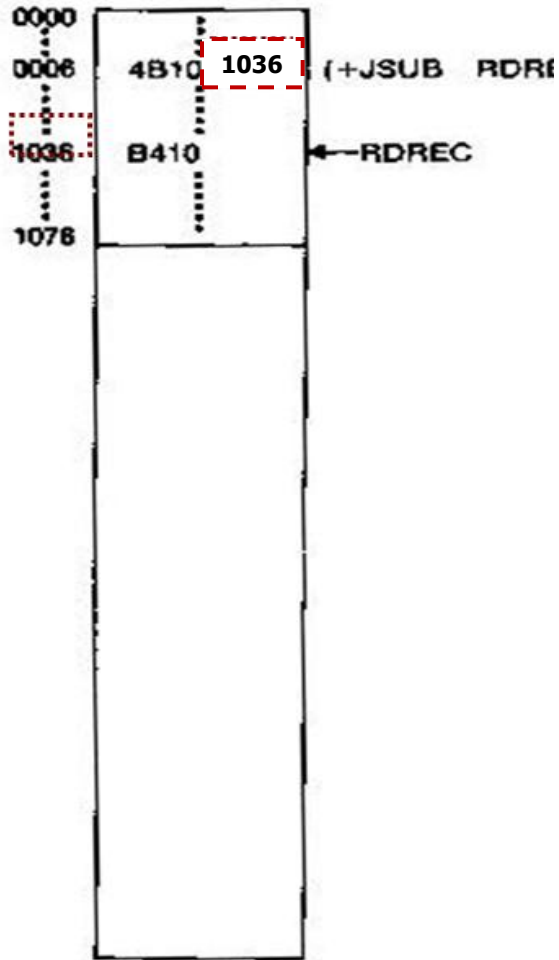


圖2.7 程式重定位範例

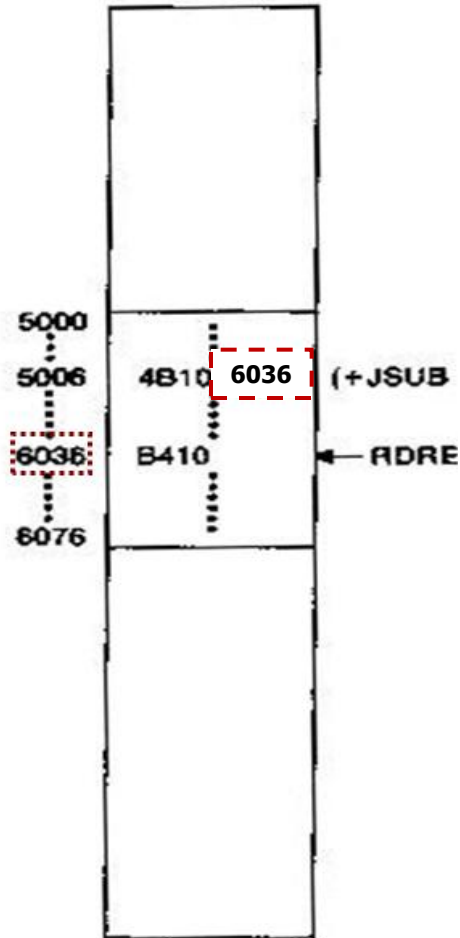


Example: relocation

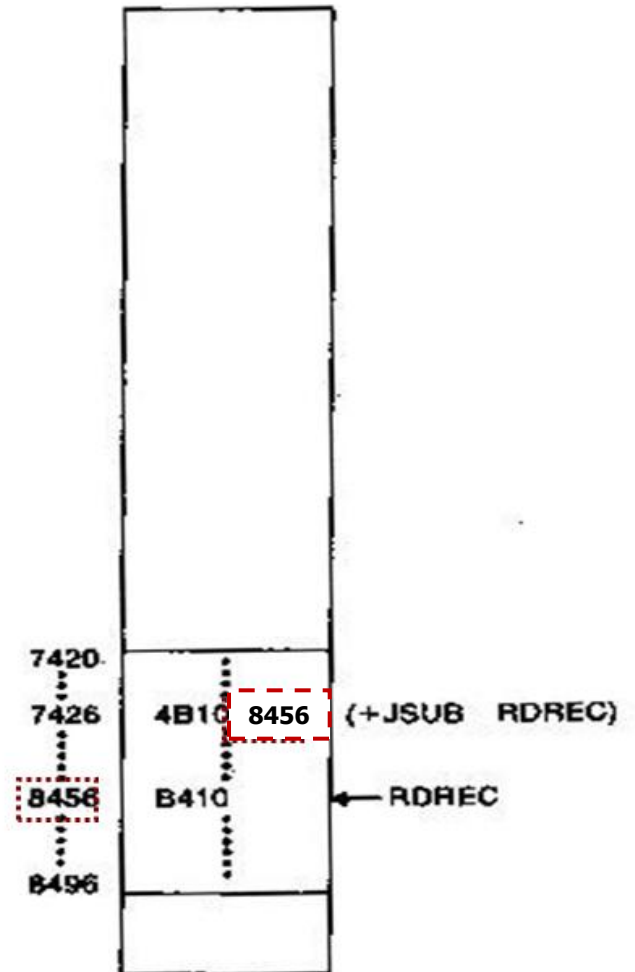
重新定址 改變位置



(a)



(b)

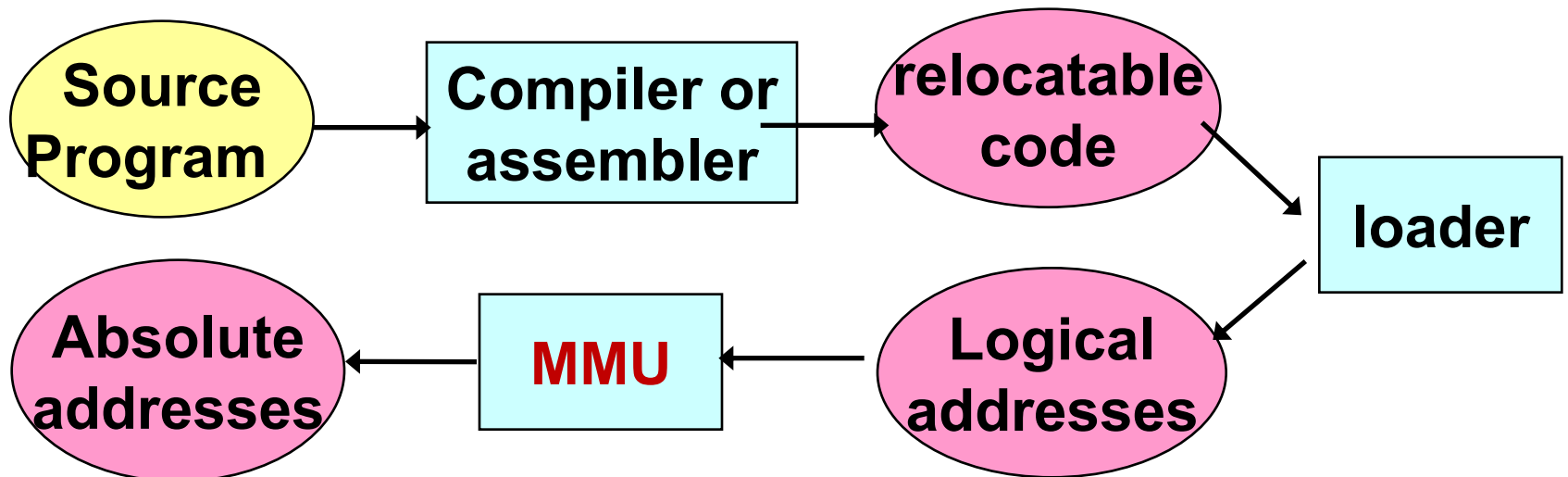


(c)

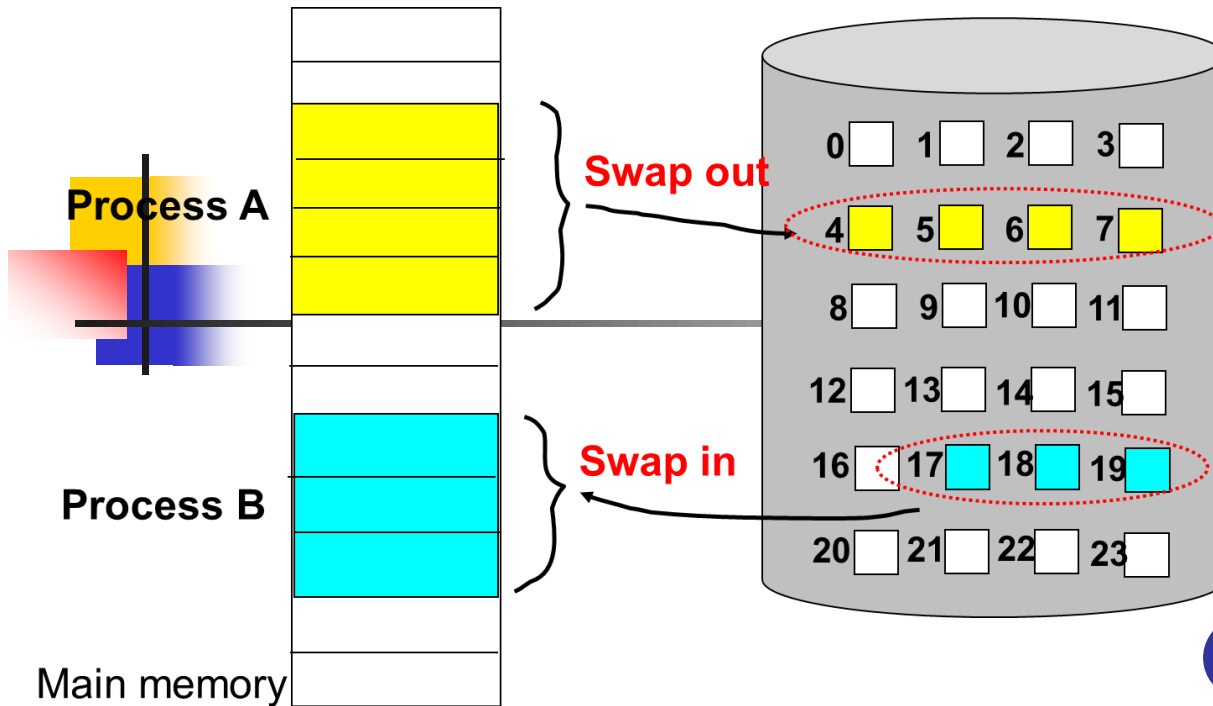


Execution-Time Address Binding

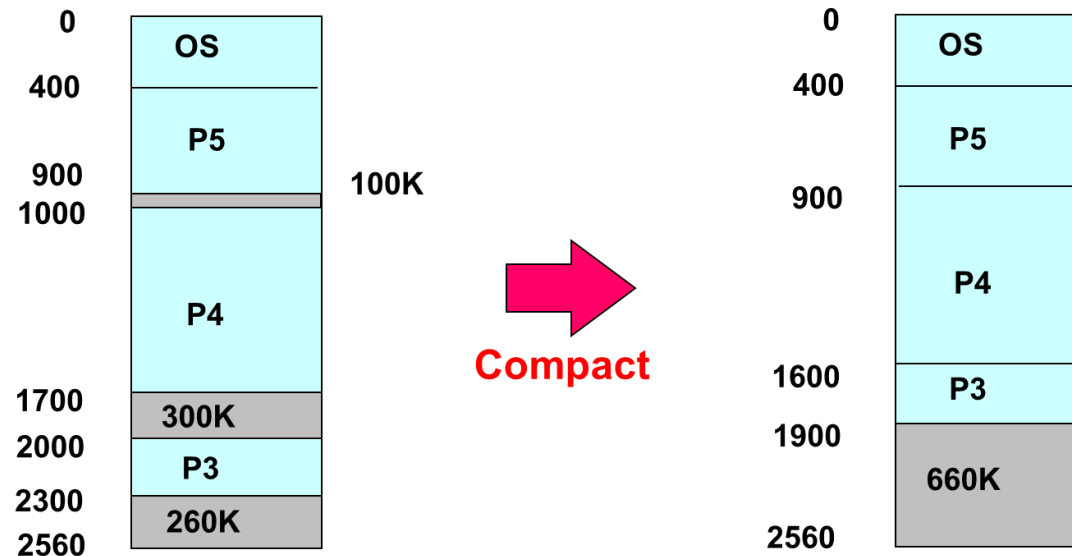
- If a process can be moved during its execution from one memory segment to another, binding must be delayed until **run time**
- Need hardware support for address maps (e.g., *base* and *limit registers*).



Swapping

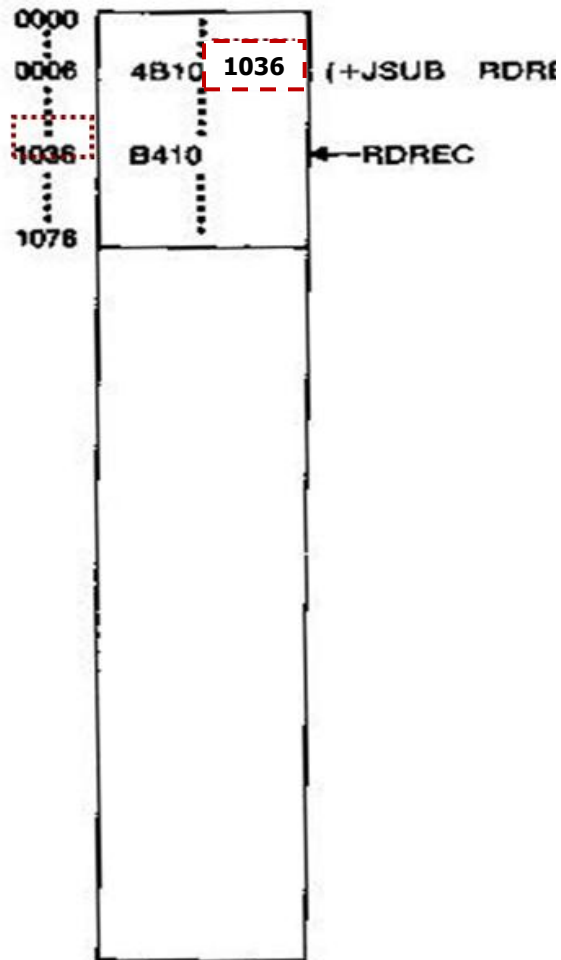


Compaction

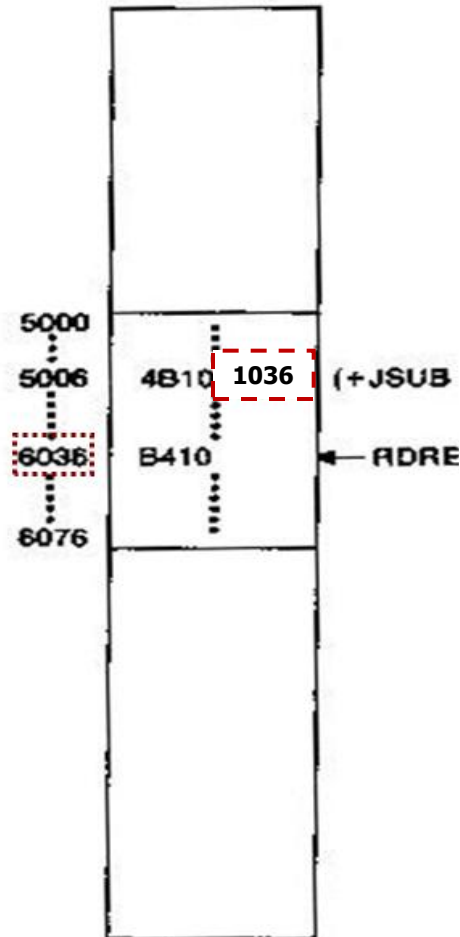


Example: dynamic relocation logical address

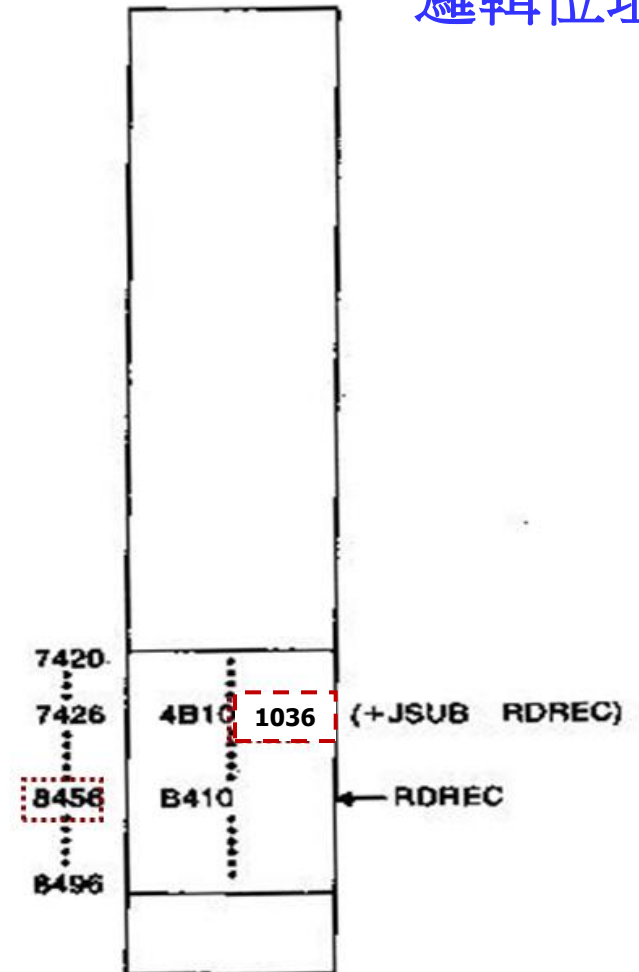
邏輯位址



(a)



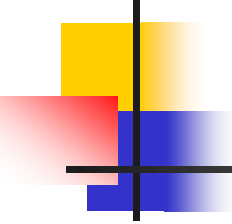
(b)



(c)



9.1.3 Logical vs. Physical Address Space



- ***Logical address***

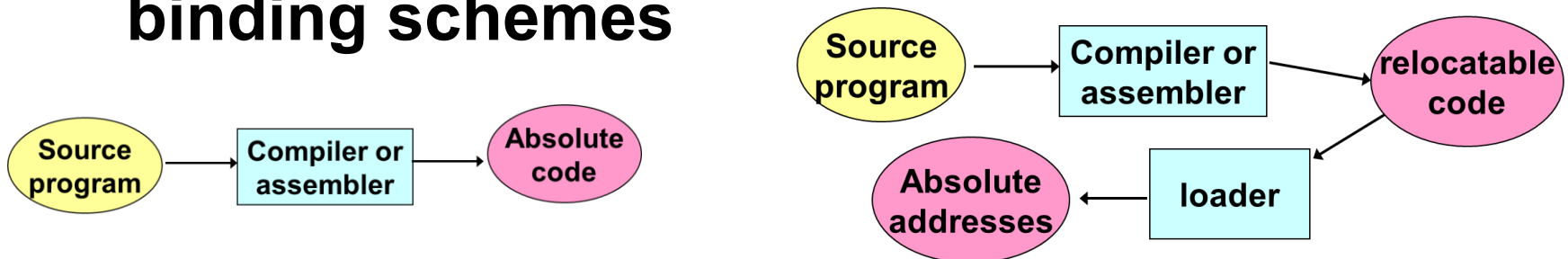
- An address generated by the CPU
- also referred to as *virtual address*
- The set of all logical addresses generated by a program is referred to as a *logical address space*.

- ***Physical address***

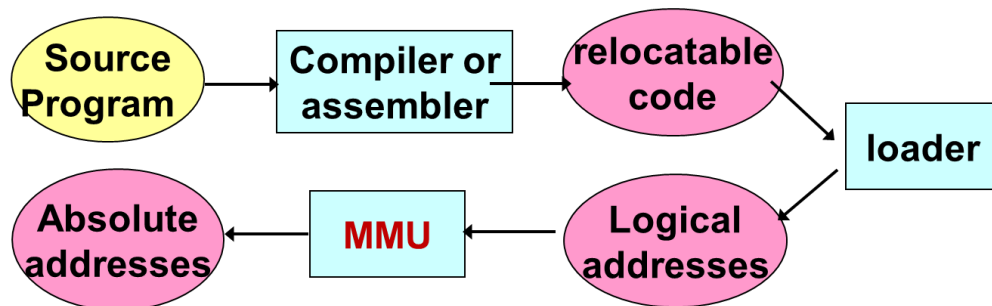
- An address seen by the memory unit
- The set of all physical addresses corresponding to these logical addresses is referred to as a *physical address space*.

Logical vs. Physical Address Space (Cont.)

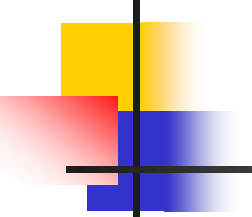
- Logical and physical addresses are the same in compile-time and load-time address-binding schemes



- logical (virtual) and physical addresses differ in execution-time address-binding scheme.



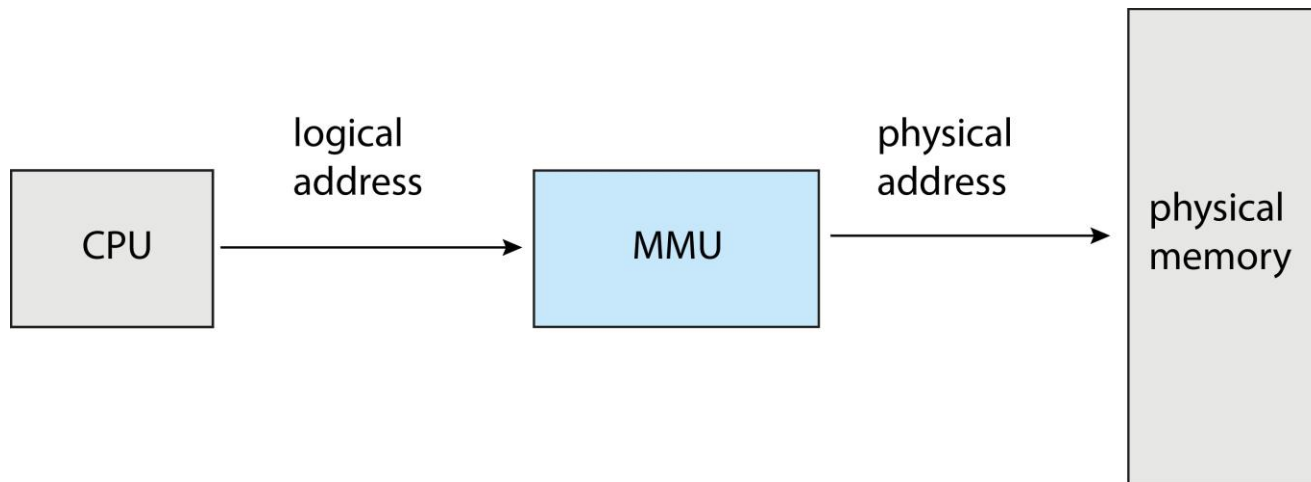
Memory-Management Unit (MMU)



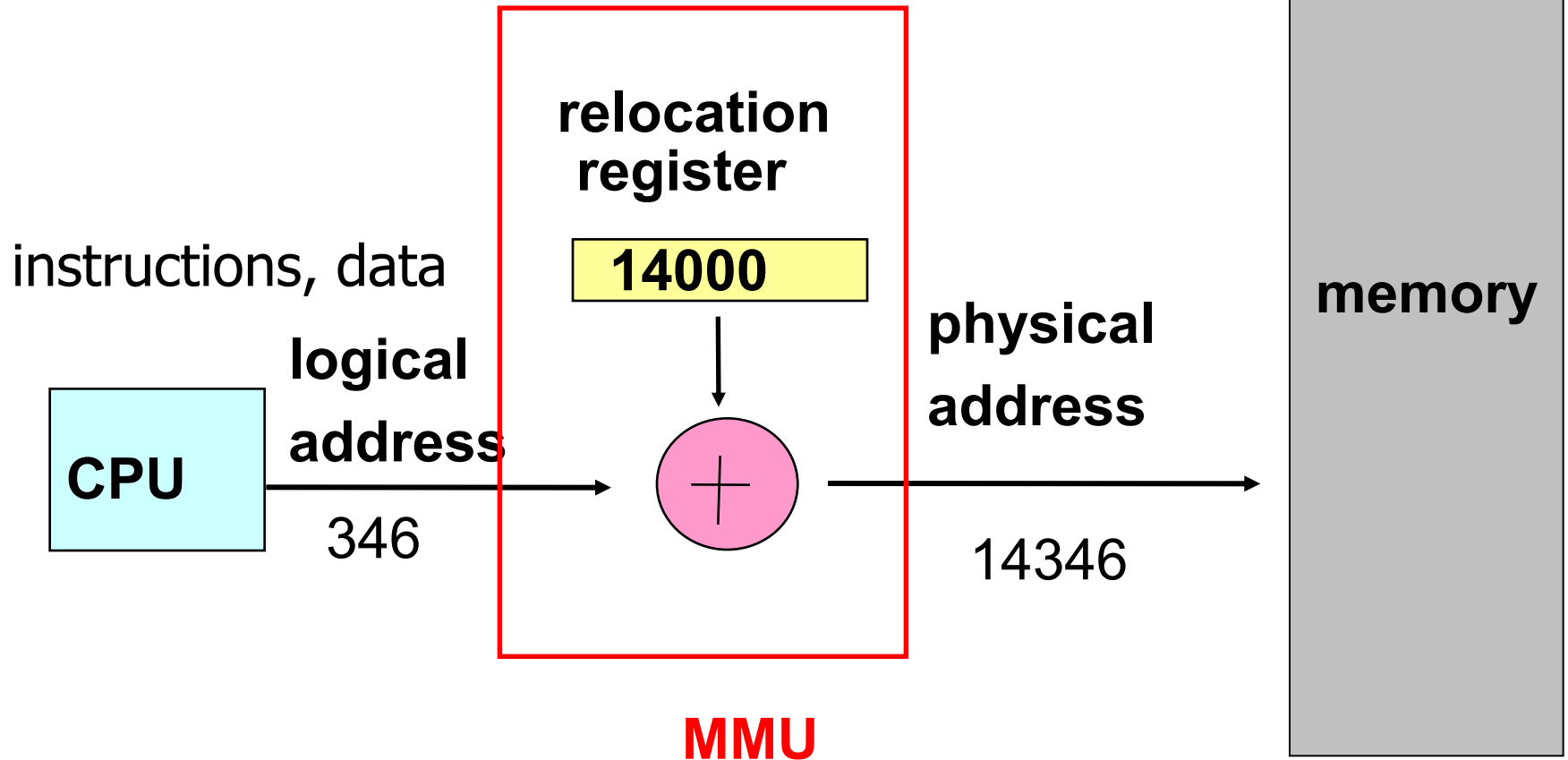
- The **run-time mapping** from virtual address to physical address is done by the **memory-management unit (MMU)**, which is a hardware device.
- A simple MMU scheme: base-register scheme
 - the value in the relocation register is added to every address generated by a user process at the time it is sent to memory. (Fig. 9.4)
- The user program deals with **logical addresses**; it never sees the *real* physical addresses.

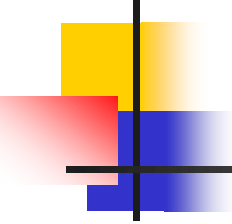
Memory-Management Unit (MMU)

- Hardware device that at run time maps virtual to physical address



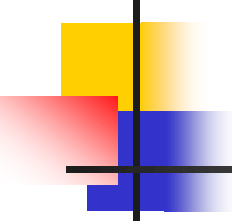
Dynamic Relocation Using a Relocation Register (Fig. 9.5)





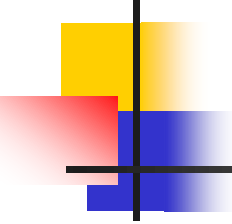
9.1.4 Dynamic Loading

- **Routine is not loaded until it is called**
- **Better memory-space utilization**
 - unused routine is never loaded
- **Useful when large amounts of code are needed to handle infrequently occurring cases**
 - e.g., error routines
- **No special support from OS is required**
- **It is the responsibility of users to design their programs to take advantage of such a method.**
- **OS may help the programmer by providing library routines to implement dynamic loading**



9.1.5 Dynamic Linking and Shared Libraries

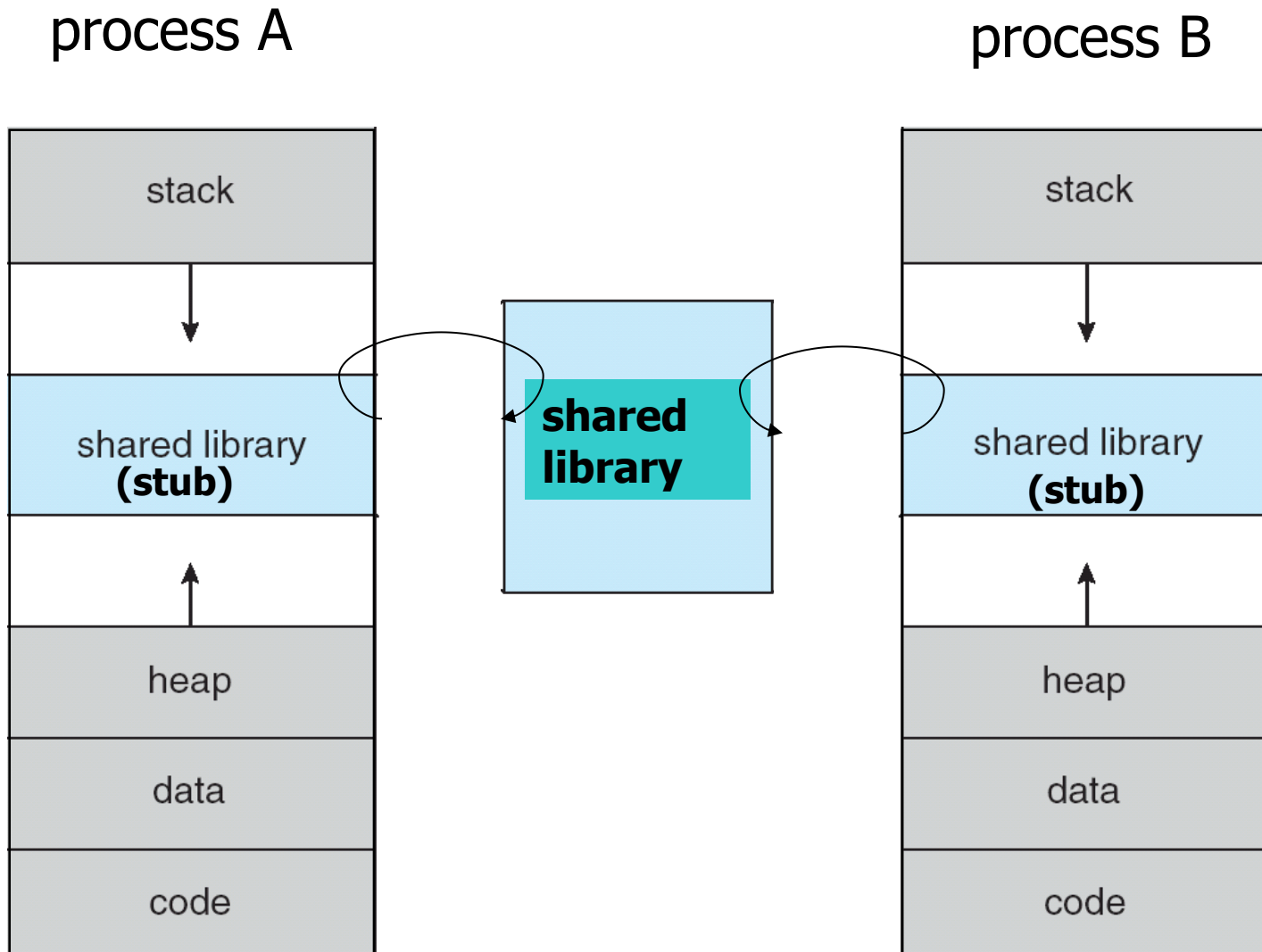
- Some OSs support only *static linking*, in which system language libraries are treated like any other object module and are combined by the loader into the binary program image

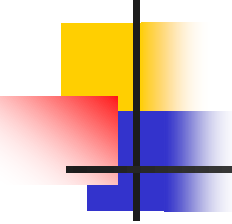


Dynamic Linking

- Linking is postponed until execution time.
- Usually used with system libraries
 - e.g., language subroutine libraries
- A **stub** is included in the executable image for each library-routine reference.
 - Small piece of code that indicates how to locate the appropriate memory-resident library routine or how to load the library if the routine is not already present.
 - Stub replaces itself with the address of the routine, and executes the routine.
 - Next time that particular code segment is reached, the library routine is executed directly, incurring no cost for dynamic linking

Linking is postponed until execution time.

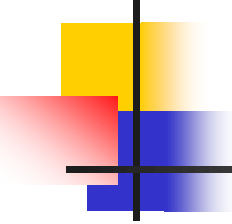




Dynamic Linking (Cont.)

- **System library can be updated without relinking all programs: *shared libraries*.**
- **Generally requires help from OS.**
 - **OS needed to check if the needed routine is in another process's memory space, or that can allow multiple processes to access the same memory addresses.**





9.2 Contiguous Memory Allocation

連續的

- Each process is contained in a single contiguous section of memory
- 9.2.1 Memory Protection
- 9.2.2 Memory Allocation
- 9.2.3 **Fragmentation**



9.2.1 Memory Protection

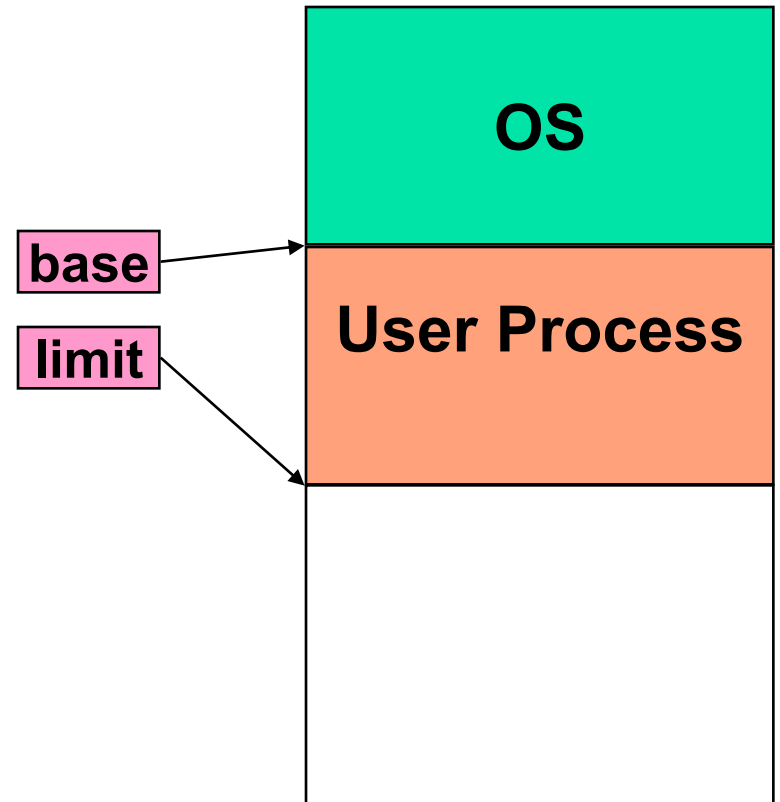
- Main memory is usually divided into two partitions:

常駐的 ■ Resident OS, usually held in low memory with interrupt vector.

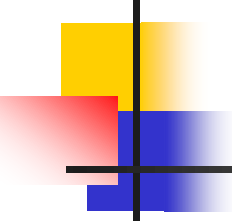
- User processes then held in high memory.

- Memory protection

- Protect the OS from user processes
- Protect user processes from one another

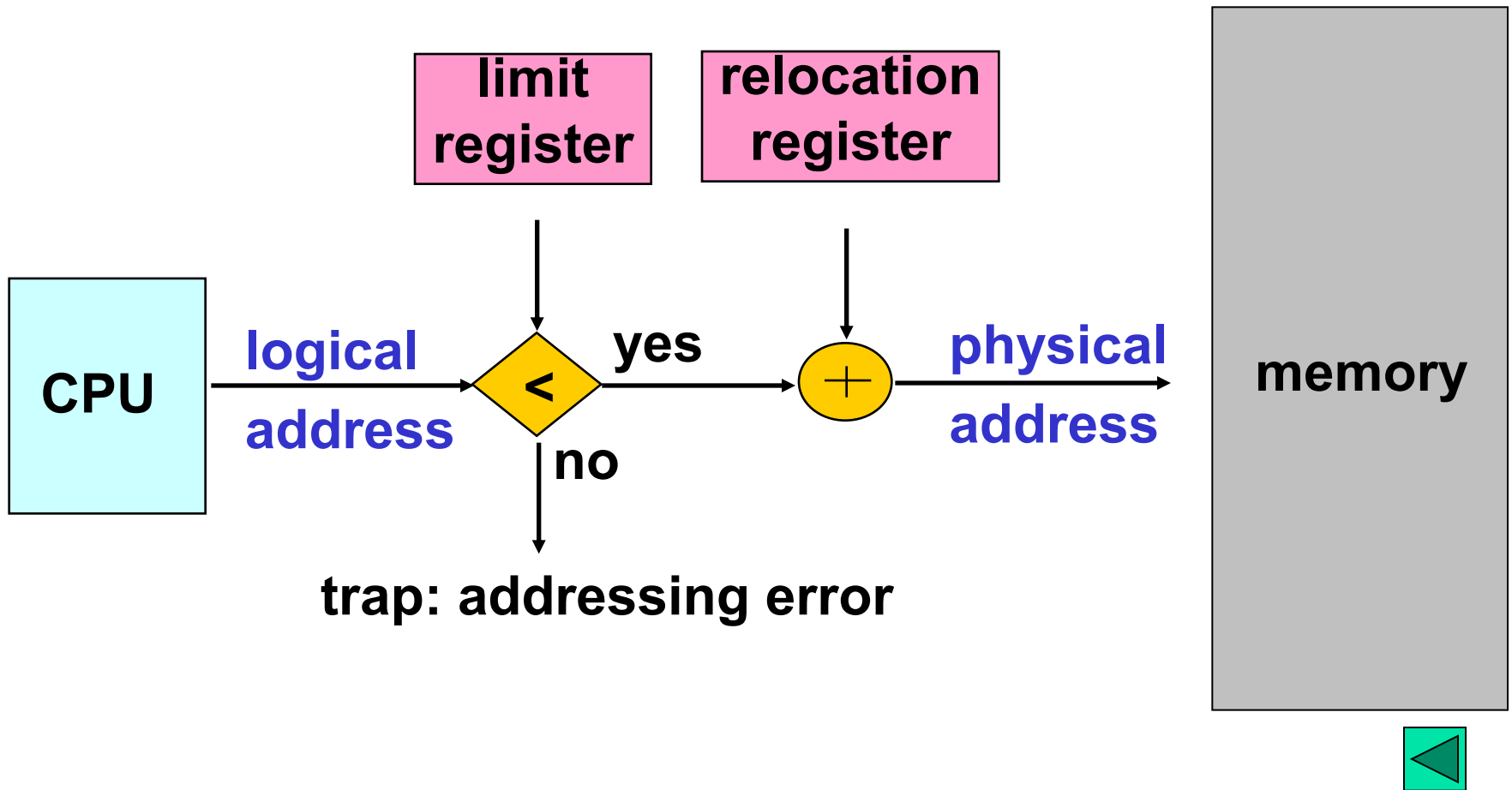


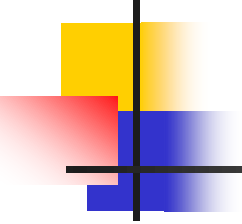
Memory Mapping and Protection (Cont.)



- **Relocation-register scheme** can be used to provide memory protection
 - **by using a relocation-register and a limit-register**
 - **relocation register** contains value of smallest physical address
 - **limit register** contains range of logical addresses
- **MMU** maps the logical address *dynamically* by adding the value in the relocation register. (Fig. 8.6)

Hardware support for relocation and limit registers (Fig. 9.6)

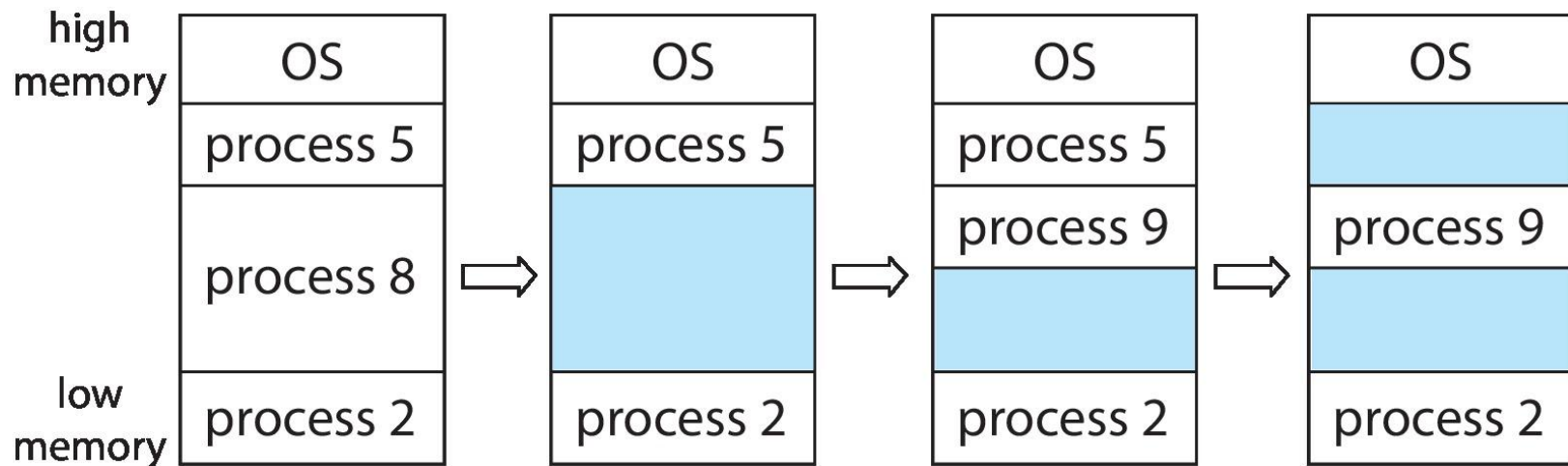




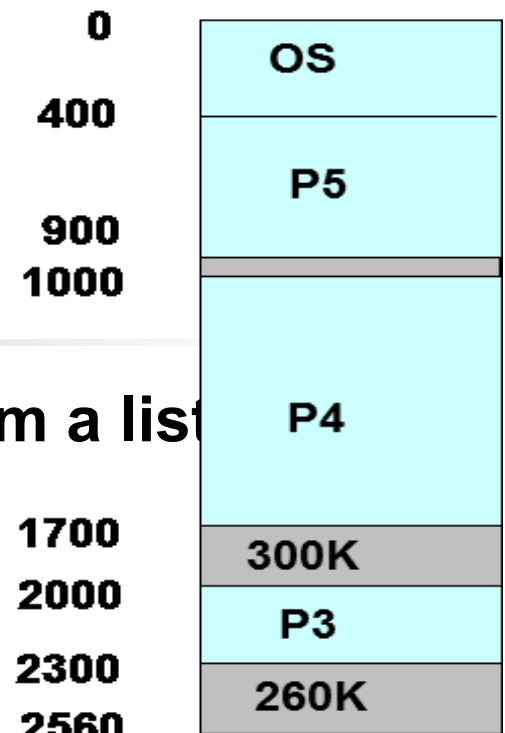
9.2.2 Memory Allocation

- **Variably Sized Partitions**
- **Variable partitions scheme**
 - OS keeps a table indicating which parts of memory are available and which are occupied.
- ***Hole***
 - One large block of available memory
 - holes of various size are scattered throughout memory.
- When a process arrives, it is allocated memory from a hole large enough to accommodate it.

Example: Variable Partition



Dynamic Storage Allocation Problem



- How to satisfy a request of size n from a list of holes
 - **first-fit**
 - allocates the *first* hole that is big enough
 - **best-fit**
 - allocates the *smallest* hole that is big enough; must search entire list, unless ordered by size
 - produces the smallest leftover hole
 - **worst-fit**
 - allocates the *largest* hole; must also search entire list
 - produces the largest leftover hole
- Simulations have shown that **first-fit** and **best-fit** are better than worst-fit in terms of **speed** and **storage utilization**.

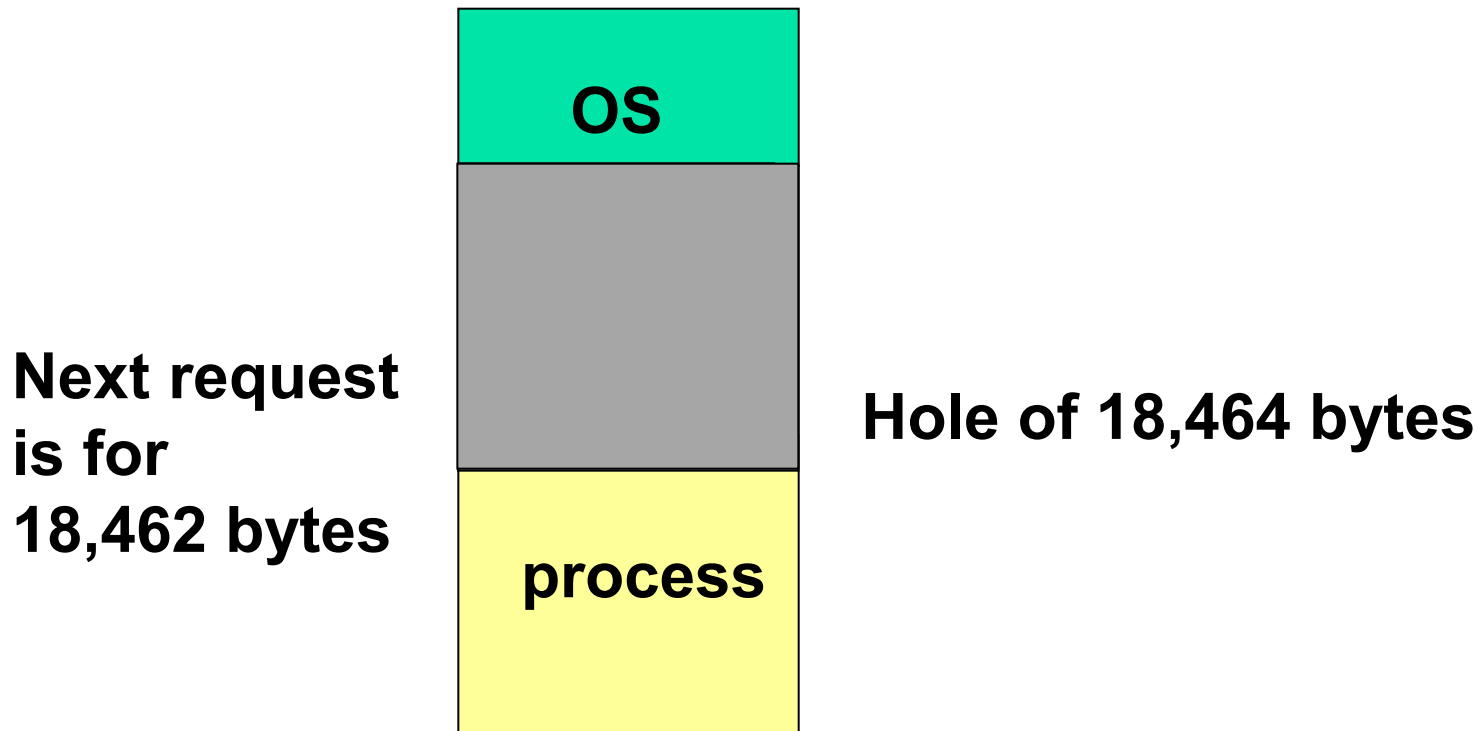


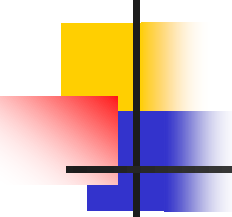
破碎

9.2.3 Fragmentation

- **External fragmentation**
 - Exists when enough total memory space exists to satisfy a request, but it is not contiguous; storage is fragmented into a large number of small holes
- **Sometimes the overhead to keep track of a hole will be substantially larger than the hole itself**
 - The general approach is to break physical memory into fixed-sized blocks and allocate memory in units based on block size.
- **Internal fragmentation**
 - allocated memory may be slightly larger than requested memory
 - unused memory that is internal to a partition

Memory allocation made in some multiple of bytes



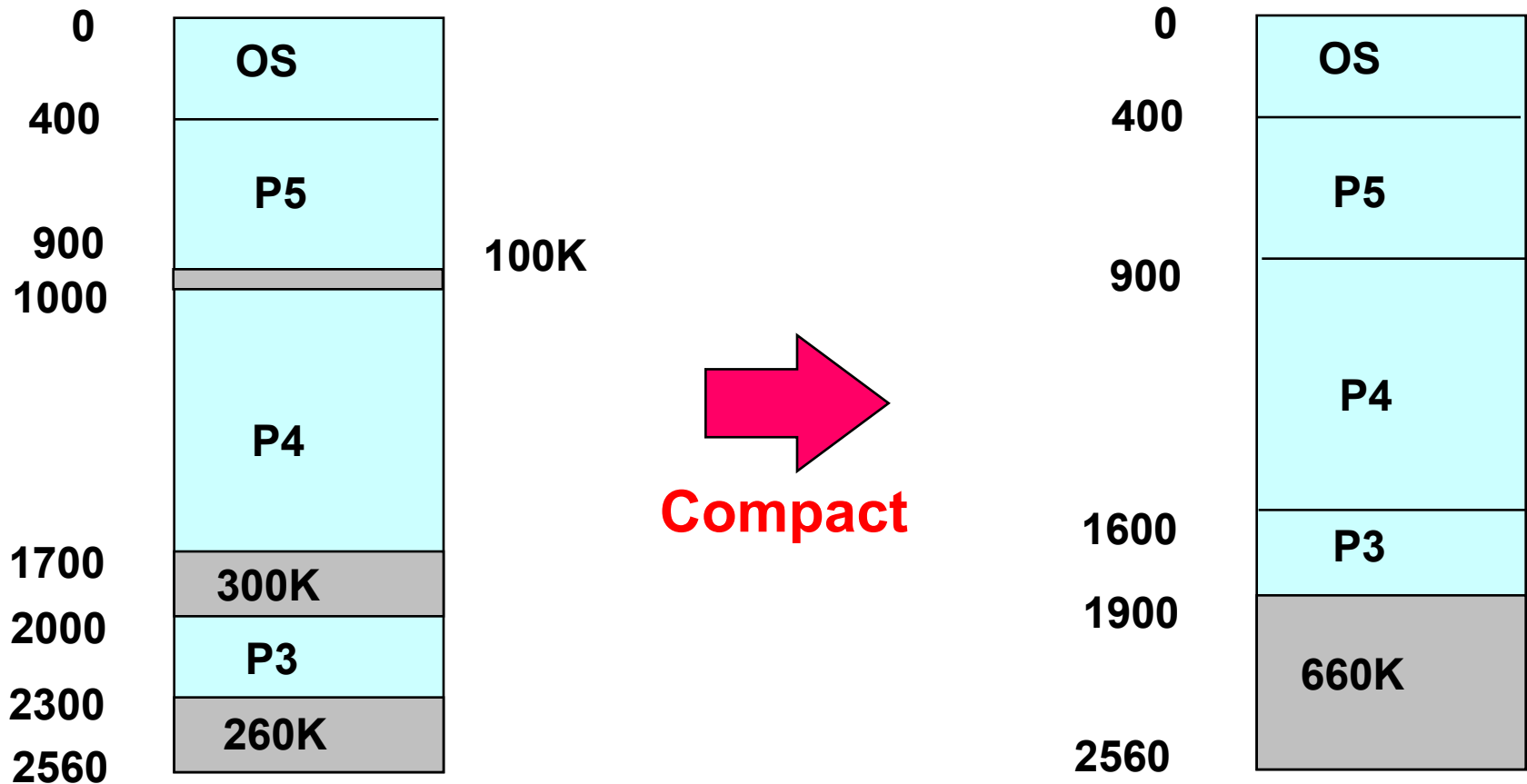


緊密 Compaction

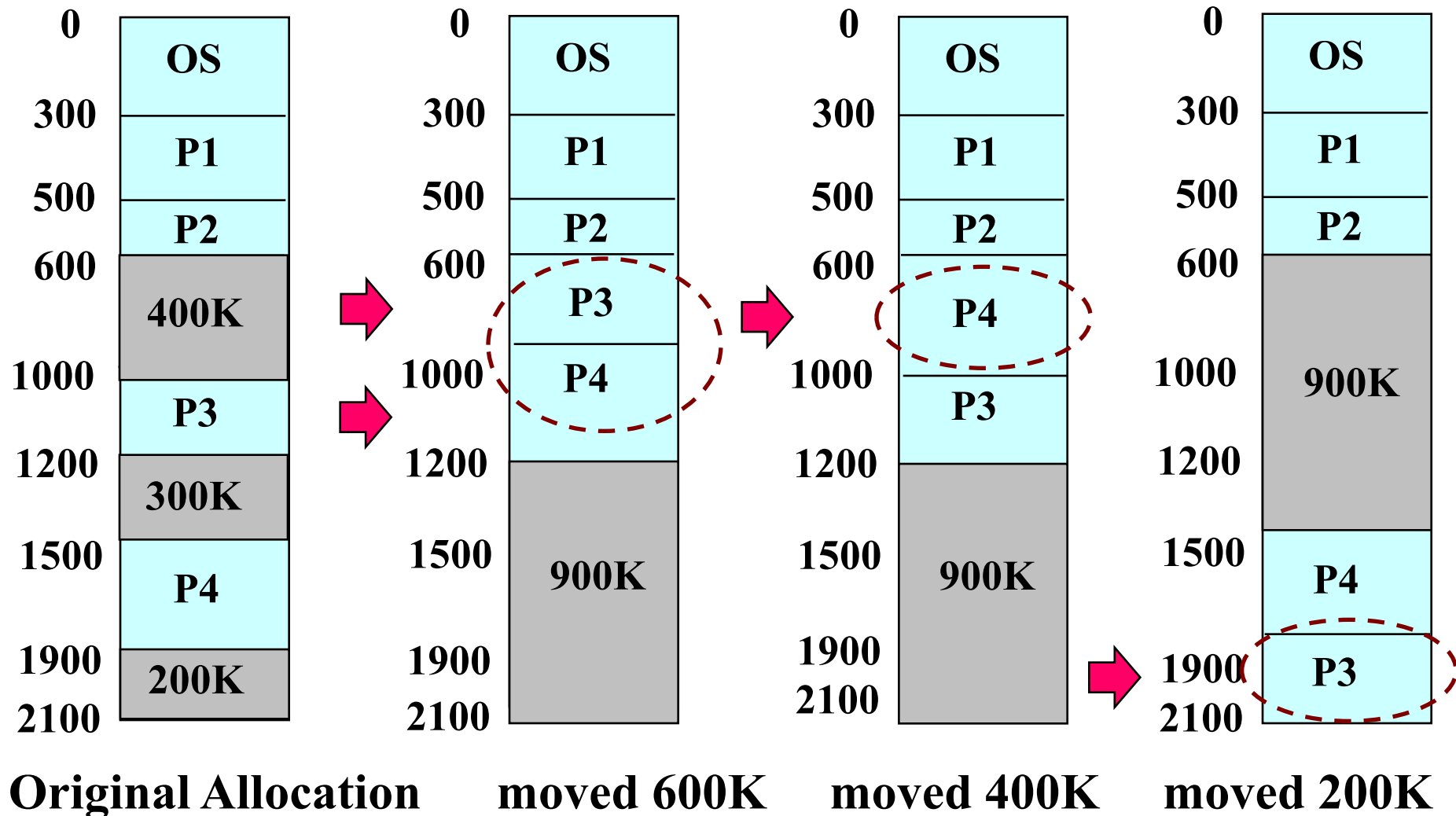
- **Reduce external fragmentation by compaction**
 - Shuffle memory contents to place all free memory together in one large block.
- **Compaction is possible *only* if relocation is dynamic, and is done at execution time.**
 - If addresses are relocated dynamically, relocation requires only moving the program and data, and then changing the base register to reflect the new base address
- **Compaction algorithms determine compaction cost**

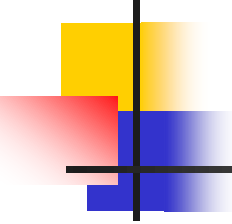
洗牌,
弄混,
改組

Compaction (Cont.)



Comparison of different ways to compact memory

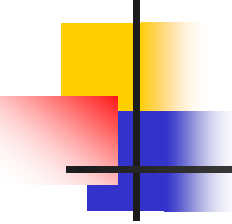




Fragmentation (Cont.)

- **Another solution to external fragmentation problem**
 - **Permit the logical-address space of the processes to be noncontiguous, thus allowing a process to be allocated physical memory whenever physical memory is available.**
 - **Paging (Section 9.3)**





9.3 Paging

- **Paging** is a memory-management scheme that permits a process's physical address space to be noncontiguous
 - Paging avoids external fragmentation and the need for compaction
 - Paging is used in most OSes
 - Implemented through cooperation between OS and H/W

 - **9.3.1 Basic Method** 
 - **9.3.2 Hardware Support** 
 - **9.3.3 Protection** 
 - **9.3.4 Shared Pages** 
- 

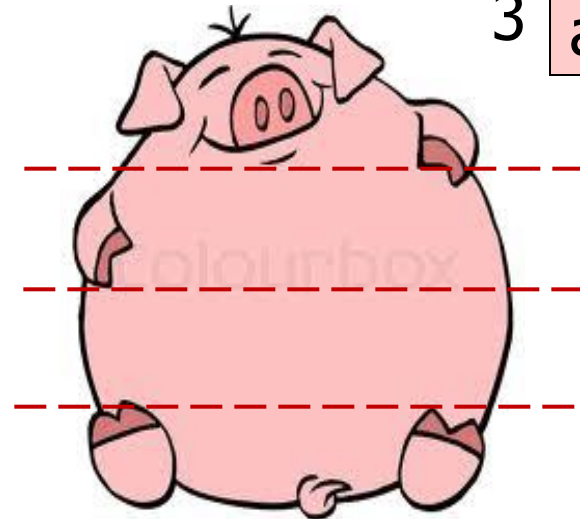


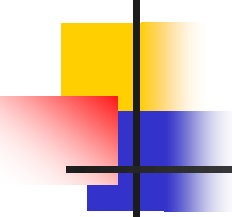
0	addr.
1	addr.



page table

0	addr.
1	addr.
2	addr.
3	addr.

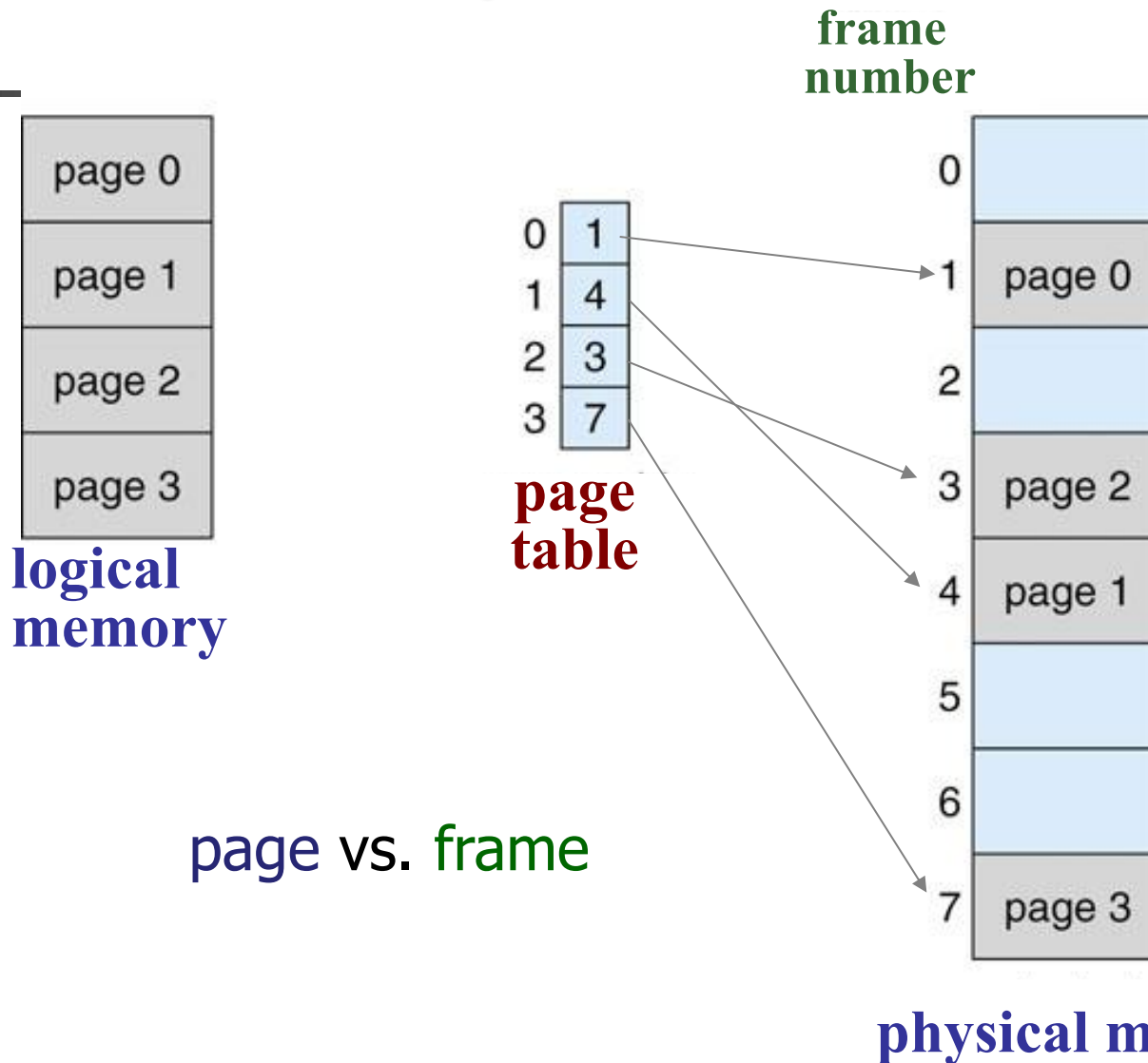




9.3.1 Paging – Basic Method

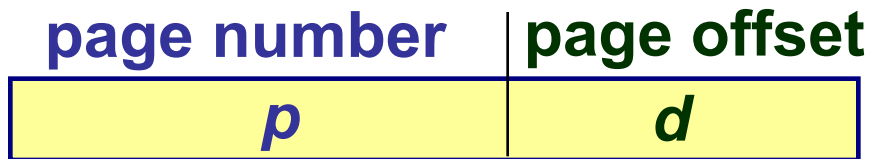
- Divide physical memory into fixed-sized blocks called **frames**
 - size is defined by hardware
 - power of 2, typically between 4 KB and 1 GB
- Divide logical memory into blocks of the same size called **pages**.
- Backing store is divided into fixed-sized blocks as the size of frames or clusters of multiple frames.
- OS keeps track of all free frames.
- To run a program of size n pages, need to find n free frames and load program.
- Set up a **page table** to translate logical to physical addresses.
- Internal fragmentation

Paging Model of Logical and Physical Memory (Fig. 9.9)



Address Translation Scheme

- Every address generated by CPU (logical address) is divided into two parts:
 - **Page number (p)**
 - used as an index into a **page table** which contains base address of each page in physical memory
 - **Page offset (d)**
 - combined with base address to define the physical memory address that is sent to memory unit



$(m - n)$ bits

n bits

page size = 2^n

logical address

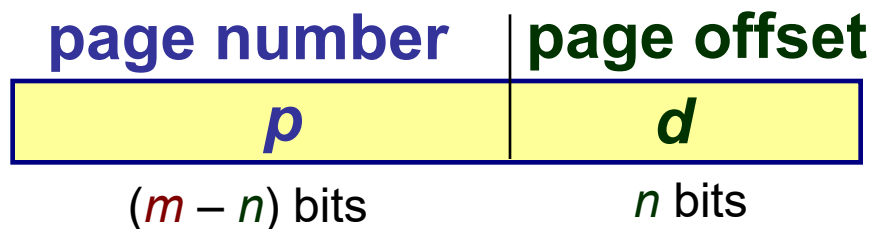
m bits

logical address space = 2^m

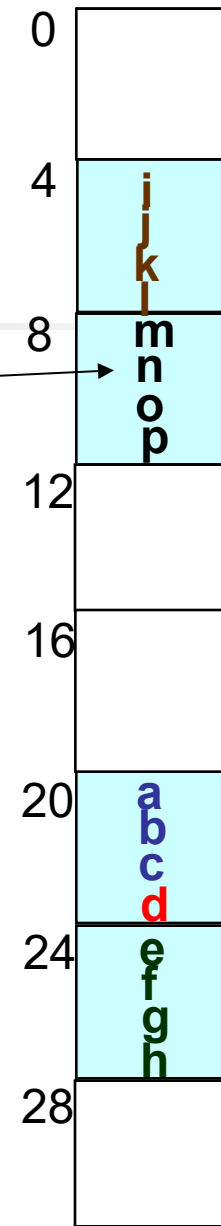
memory address

address 9
= 01001_2
= 2^{th} page, 1^{st} byte

address
= p^{th} page, d^{th} byte



page size = 2^n

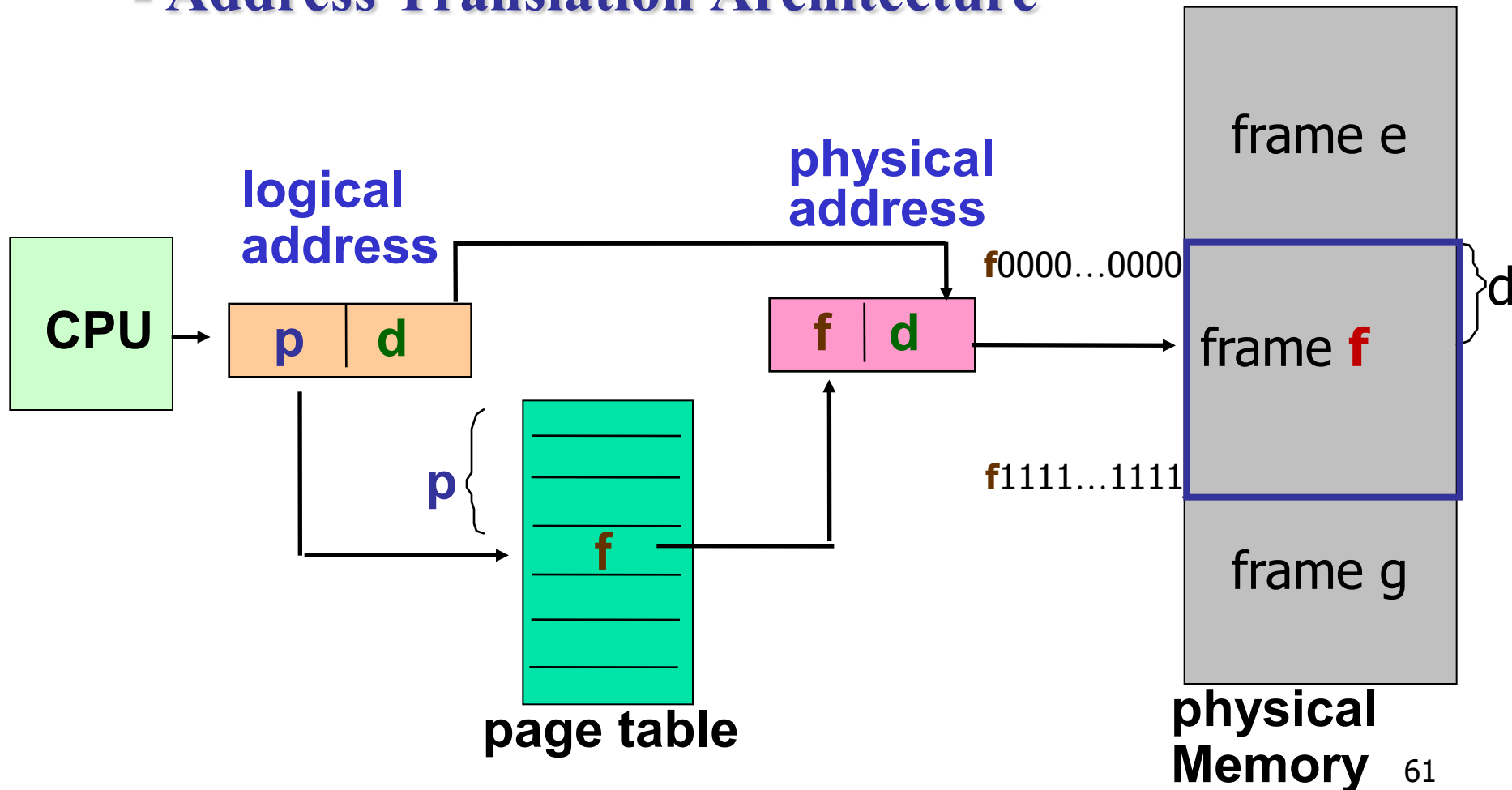


physical memory

位址	10進位	2進位	addr bits=4 =2 ⁴ bits 0~15
	0	0000	
	1	0001	
	2	0010	
	3	0011	
	4	0100	page size 2 ² bytes
	5	0101	
	6	0110	
	7	0111	
	8	1000	
	9	1001	
	10	1010	
	11	1011	
	12	1100	
	13	1101	
	14	1110	
	15	1111	

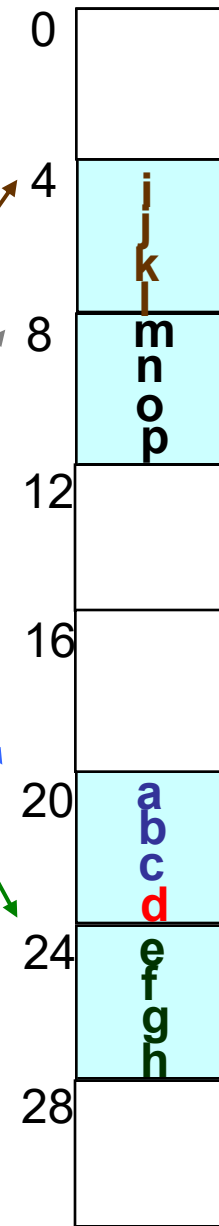
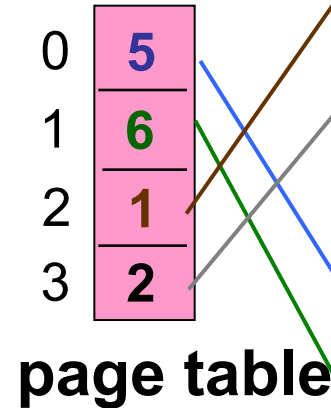
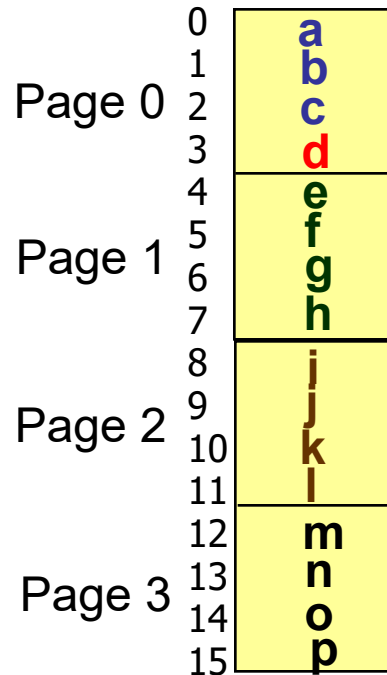
Paging Hardware

- Hardware support for paging (Fig. 9.8)
- Address Translation Architecture



Paging Example (Fig. 9.10)

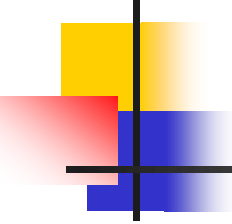
Memory Size:
32 Bytes
Page Size: 4 Bytes



logical addr. **00011**

phy. addr. **10111**

Logical address **3** (page 0, offset **3**)
maps to physical addr. 23 ($= (5 * 4) + 3$)



Paging (Cont.)

- **Paging itself is a form of dynamic relocation**
- **No external fragmentation**
 - Any free frame can be allocated to a process that needs it.
- **May have some internal fragmentation**
 - last frame
 - Average $\frac{1}{2}$ page per process
- **Page size tradeoff**
 - Large page size $\Rightarrow$ may have large internal fragmentation
 - Small page size $\Rightarrow$ more page-table entry overhead, disk I/O is less efficient
 - Typically either 4KB or 8 KB
 - Some CPUs and kernels even support multiple page sizes

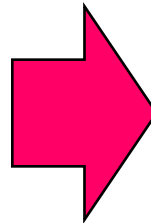
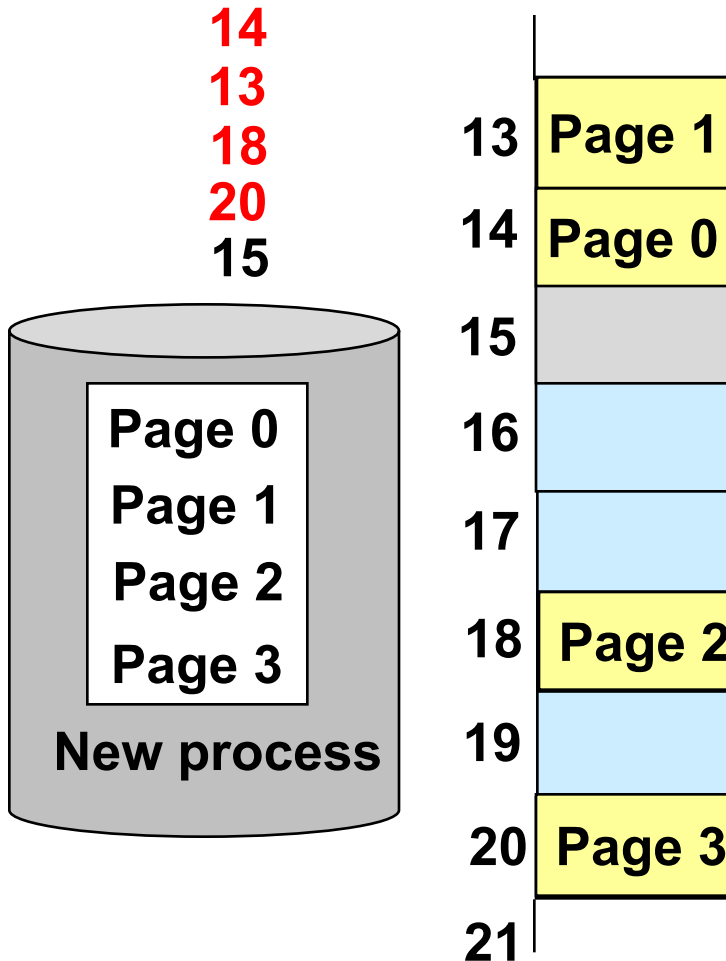


Paging (Cont.)

- OS manage physical memory and must keep track of all free frames.
 - A **frame table** has one entry per physical page frame
 - To run a program of size n pages, need to find n free frames and load program.
- OS must control the mapping
 - Maintains a copy of page table for each process
 - Set up the page table for address-translation hardware to translate logical to physical addresses.
 - Paging increases the context-switch time.

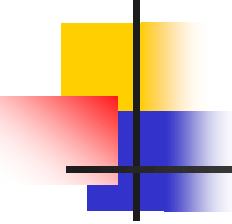
Free Frames Allocation (Fig. 9.11)

free-frame list



0	14
1	13
2	18
3	20

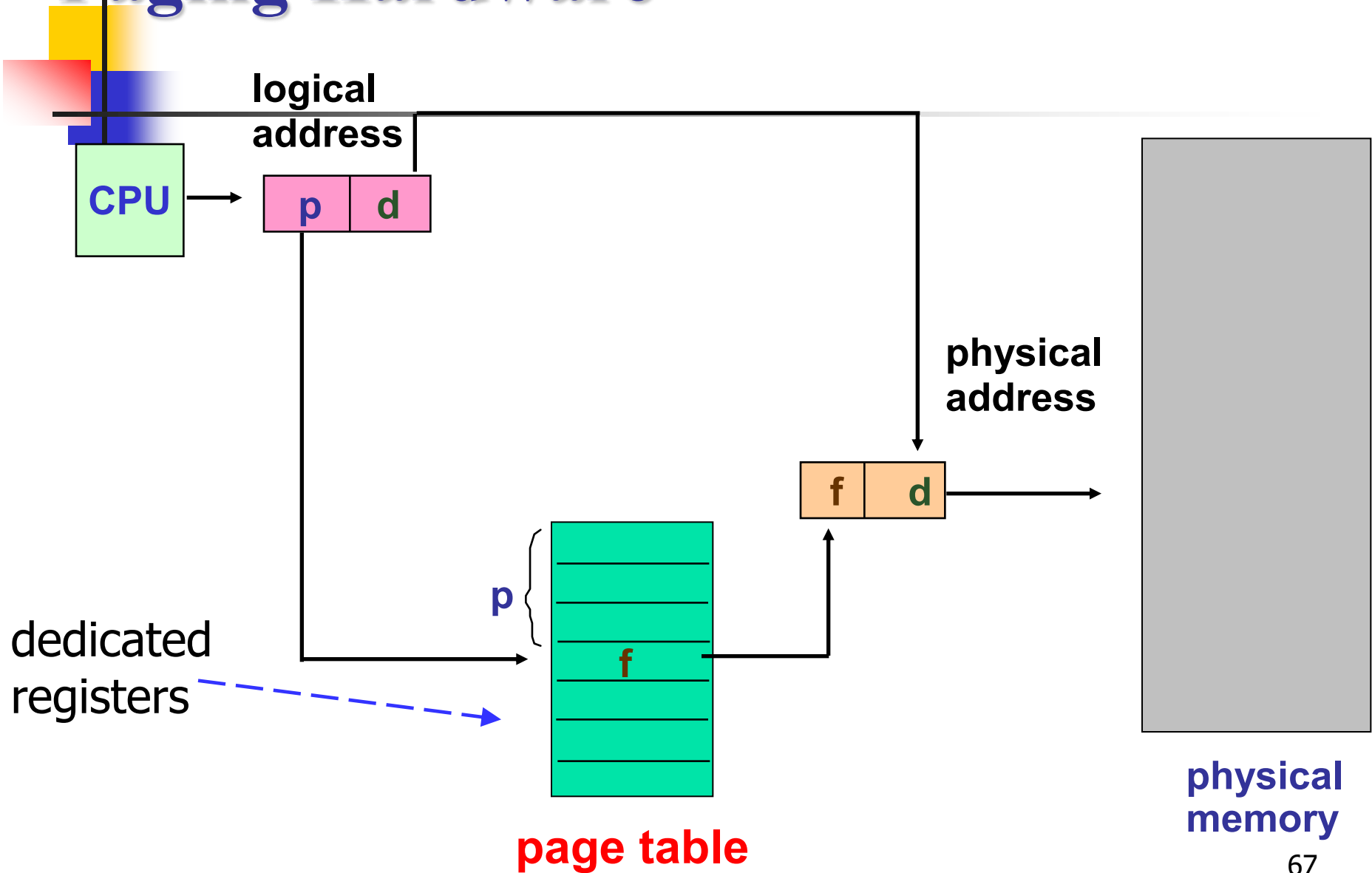
new-process **page table**



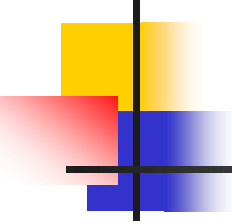
9.3.2 Hardware Support

- Hardware implementation of page table can be done in many different ways:
 - Implemented as a set of dedicated **registers**
 - fast but small size
 - Page table is kept in **main memory**, and a *page-table base register (PTBR)* points to the page table.
 - Page table can be very large
 - Changing page tables requires changing this one register, substantially reducing context-switch time

Paging Hardware

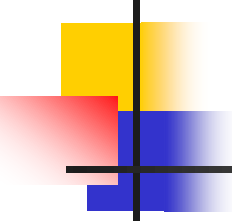






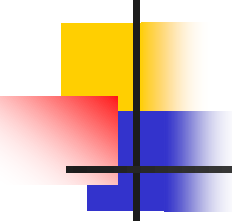
Implementation of Page Table (Cont.)

- Problem when page table being kept in **main memory**
 - 2 memory accesses is needed for every data/instruction access
 - One for the page table and one for the data/instruction.
 - Too slow
- Standard solution
 - Use a special, small, fast-lookup hardware cache, called translation look-aside buffer (TLB) which caches some page-table entries
 - Fast but expensive



Translation Look-aside Buffers

- The TLB is associative, high-speed memory.
- Each entry in the TLB consists of a key and a value
- When presented with an item, it is compared with all keys simultaneously
 - If found, value is returned
- Search is fast (hardware) but expensive
- Typically, 32-1024 entries in a TLB



TLB (Cont.)

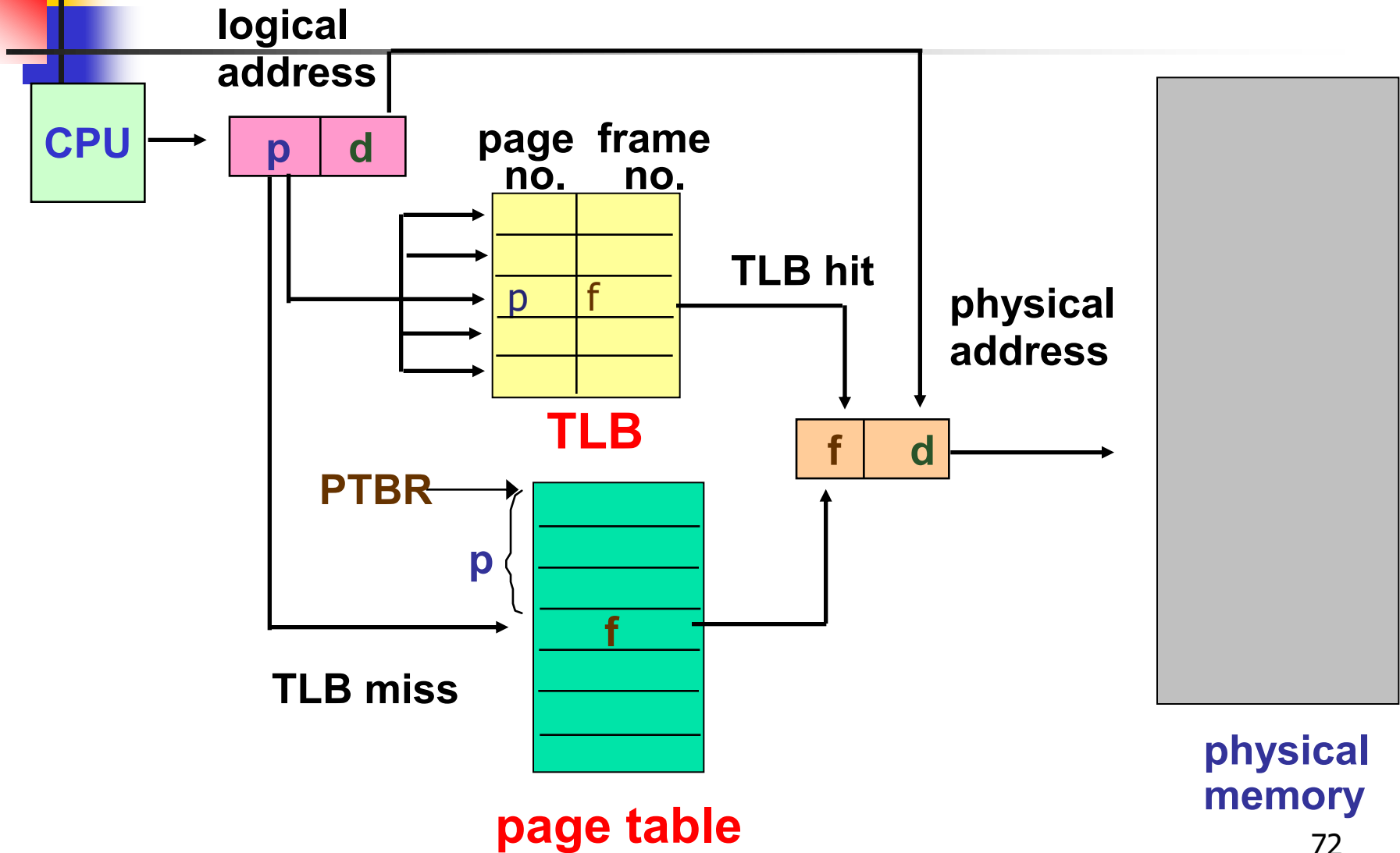
- Associative memory – parallel search

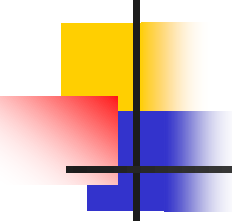
Page #	Frame #

Address translation (A' , A'')

- If A' is in associative register, get frame # out.
- Otherwise get frame # from page table in memory

Paging Hardware with TLB (Fig. 9.12)



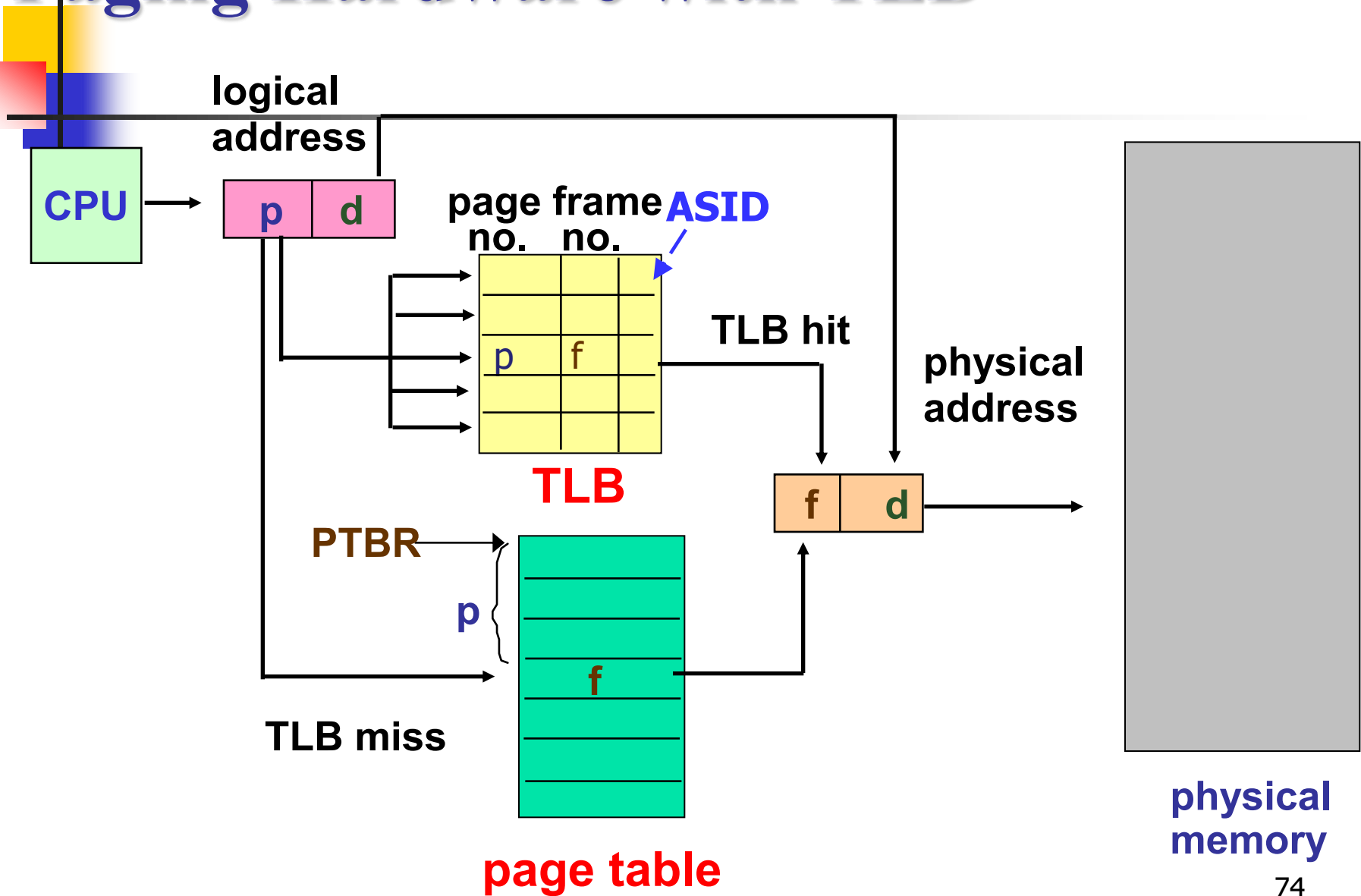


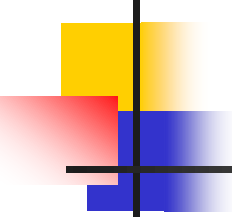
Translation Look-aside Buffers

(Cont.)

- Replacement policies since number of TLB entries are limited
 - maintain the TLB entries by H/W or OS
 - When context switch (a new page table is selected), the TLB must be *flushed* (erased) to avoid wrong translation
- Some TLBs store **address-space identifiers (ASIDs)** in each TLB entry.
 - An ASID uniquely identifies each process and is used to provide address space protection for that process.
 - Allows the TLB to contain entries for several different processes simultaneously.

Paging Hardware with TLB

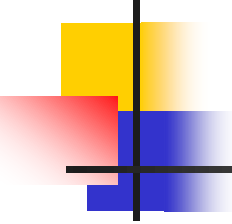




Effective Memory-Access Time

- A TLB lookup in modern HW is part of the instruction pipeline, essentially adding no performance penalty
- **Hit Ratio**, the percentage of times that a page number is found in the TLB
- Assume:
 - Memory cycle time is **ma** time unit
 - Hit ratio = α
- **Effective memory-access time (EAT)**

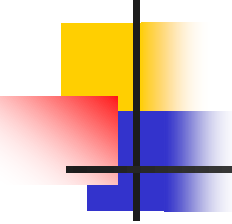
$$\text{EAT} = \underbrace{(ma)}_{\text{TLB hit}} \alpha + \underbrace{(2ma)}_{\text{TLB miss}} (1 - \alpha)$$



Effective Access Time (Cont.)

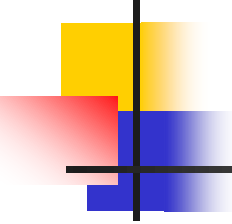
- e.g., $ma=10$ nanosec., $\epsilon=0$, hit ratio= 80%
 - $EAT= 10 * 0.8 + 20 * 0.2 = 12$ nanosec.
 - 20% slowdown in average memory-access time (from 10 to 12)
- e.g., $ma=10$ nanosec., $\epsilon=0$, hit ratio= 99%
 - $EAT= 10 * 0.99 + 20 * 0.01 = 10.1$ nanosec.
 - 1% slowdown in average memory-access time (from 10 to 10.1)





9.3.3 Protection

- Memory protection in a paged environment is accomplished by associating **protection bits** with each frame.
 - Read / Write, Read-only, Execution-only
 - Normally, these bits are kept in **page table**
 - Illegal attempts cause hardware **trap** to OS (memory-protection violation)



Protection (Cont.)

- **Valid-invalid** bit attached to each entry in the page table: (Fig. 9.13)
 - valid: the associated page is in the process's logical address space, and is thus a legal page (or legal).
 - invalid: the page is not in the process's logical address space.
 - OS sets this bit for each page to allow or disallow accesses to that page.
 - Illegal addresses cause hardware trap to OS
- Some systems provide hardware, in the form of a **page-table length register (PTLR)**
 - Indicate the size of the page table
 - Is checked against every logical address to verify that the address is in the valid range for the process

Valid or invalid bit in a page table (Fig. 9.13)

- A **valid-invalid** bit is attached to each page table entry
- invalid: the page is not in the process's logical address space

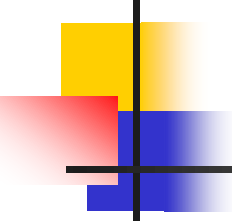
00000	Page 0
	Page 1
	Page 2
	Page 3
	Page 4
10468	Page 5
12287	

logical memory

frame No.	valid/ invalid bit	
0	2	v
1	3	v
2	4	v
3	7	v
4	8	v
5	9	v
6	0	i
7	0	i

page table

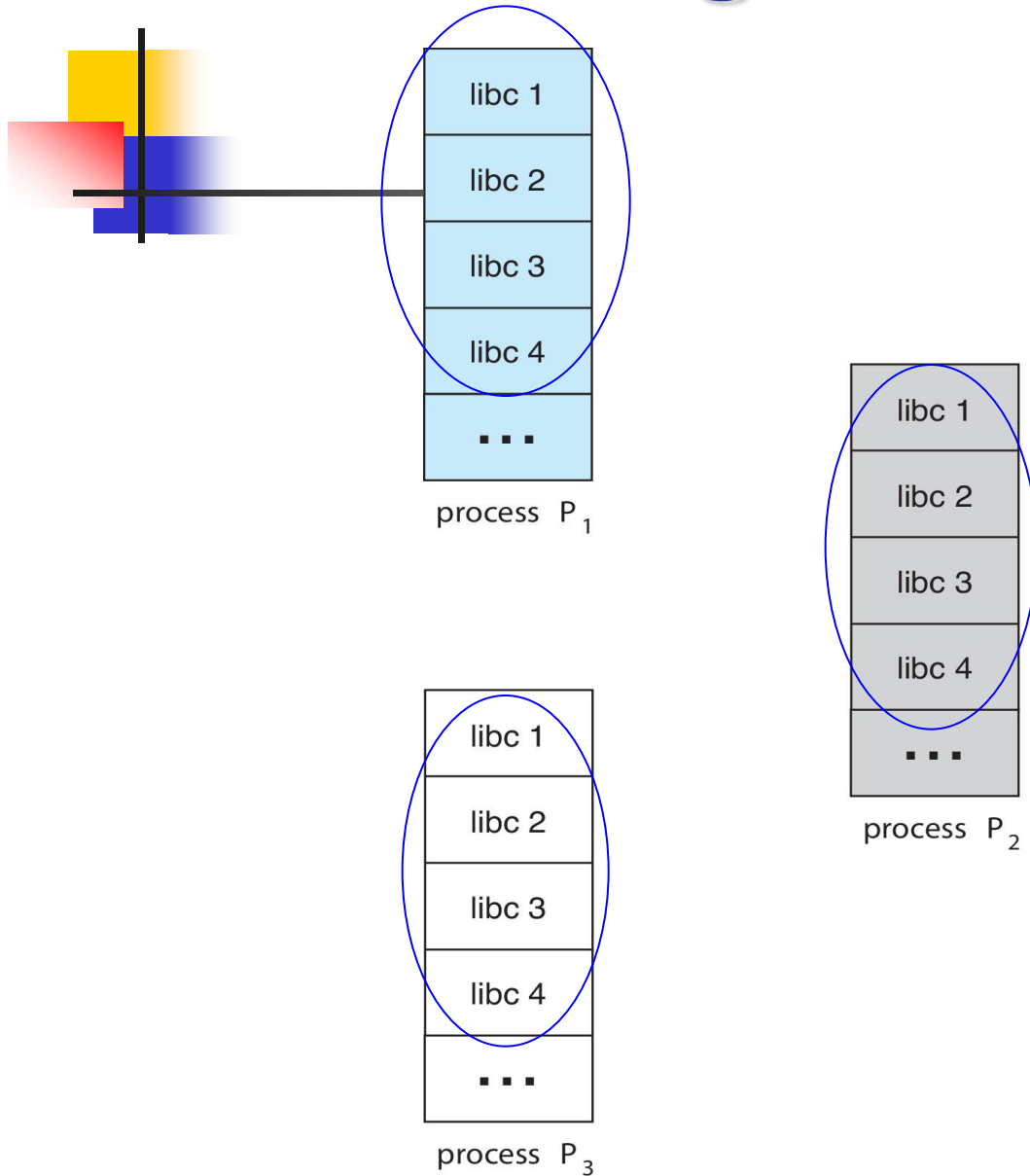
0	
1	
2	Page 0
3	Page 1
4	Page 2
5	
6	
7	Page 3
8	Page 4
9	Page 5
:	
n	



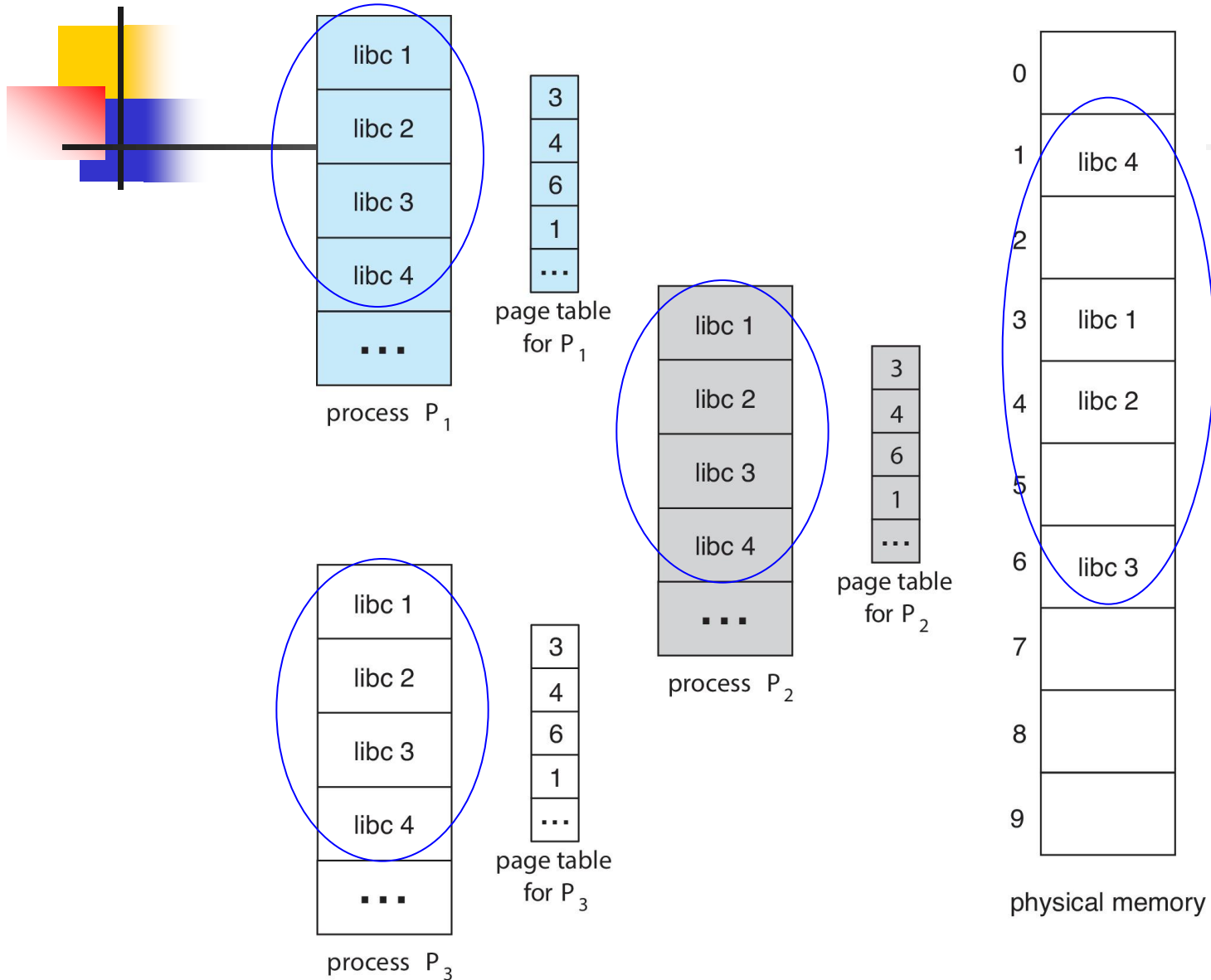
9.3.4 Shared Pages

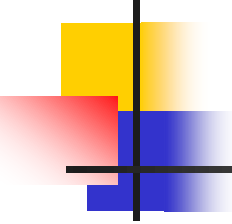
- Another advantage of paging is the possibility of **sharing common code** (must be read only “**reentrant code**”).
 - ***reentrant code (or pure code)***: non-self-modifying code (code that never changes during execution)
- Only one copy of the shared code (i.e., compilers, window systems, run-time libraries, database systems) needs to be kept in physical memory.
- Some OSes implement shared memory using shared pages.

Shared Pages Example (Fig. 9.14)



Shared Pages Example (Fig. 9.14)

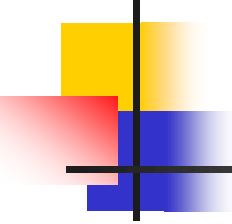




9.4 Structure of the Page Table

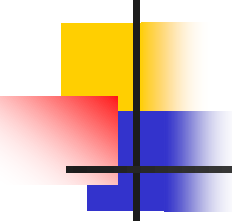
- **Some of the most common techniques for structuring the page table:**
 - **9.4.1 Hierarchical Paging** 
 - **9.4.2 Hashed Page Tables** 
 - **9.4.3 Inverted Page Tables** 
 - **9.4.4 Oracle SPARC Solaris** 





9.4.1 Hierarchical Paging

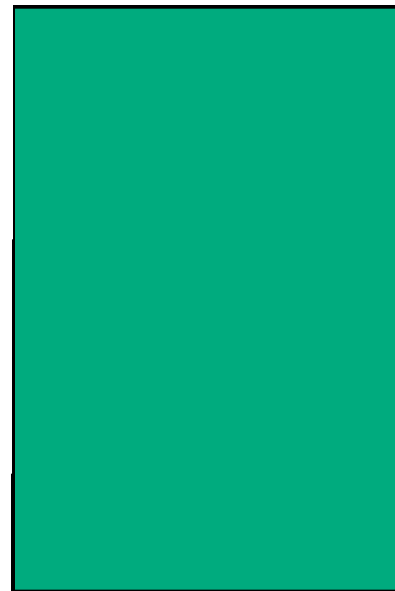
- Most modern computer systems supports a large logical address space (2^{32} to 2^{64}).
- Problem: the **page table becomes very large**.
 - e.g., a 32-bit logical address space and page size = 4 K (2^{12}), then we have **1 million entries**.
 $\frac{2^{32}}{2^{12}} = 2^{20}$
 - If each entry 4 B, each process may need 4MB memory for page table
- Solution: divide the page table into smaller pieces for not to allocate page table contiguously in the main memory.
- A **two-level paging scheme** can be used, in which the **page table itself is also paged**. (Fig. 9.15)
 - e.g., Pentium II, VAX architecture



Two-Level Paging

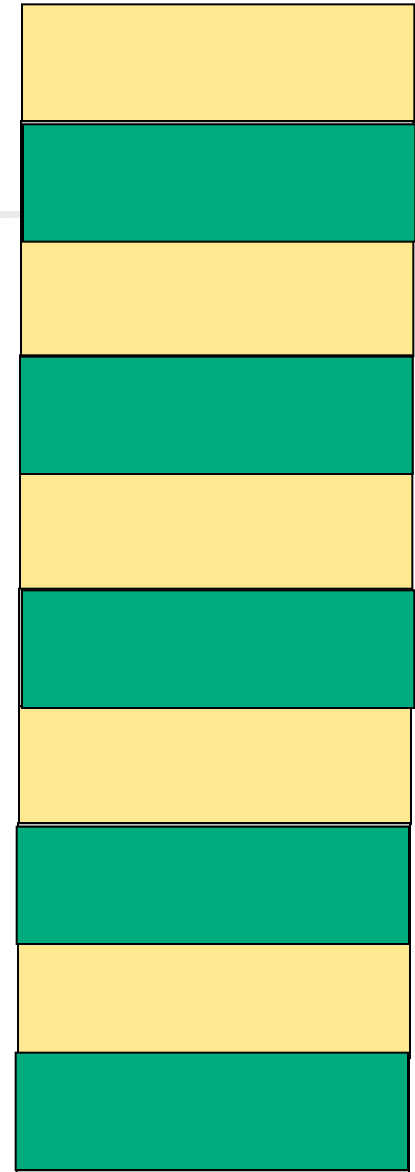
When page table
is too large

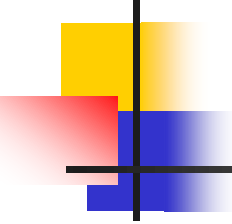
=> Page
the page
table itself



Page table

RAM





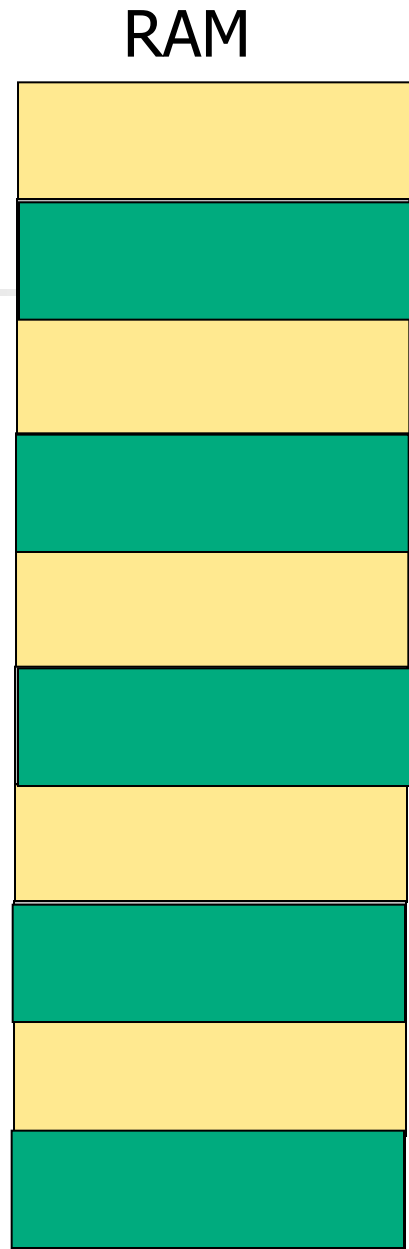
Two-Level Paging



outer page table

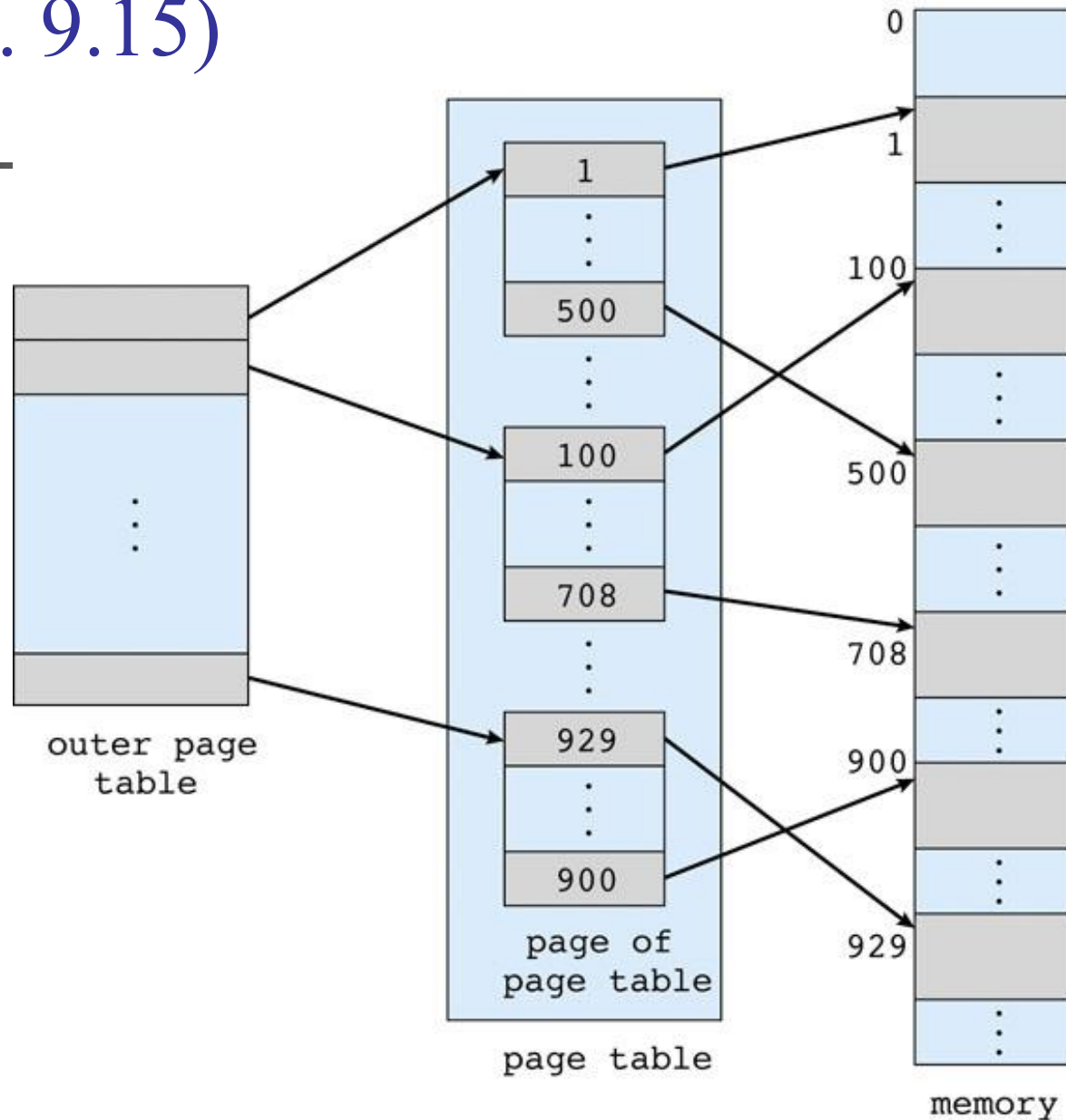


Page table



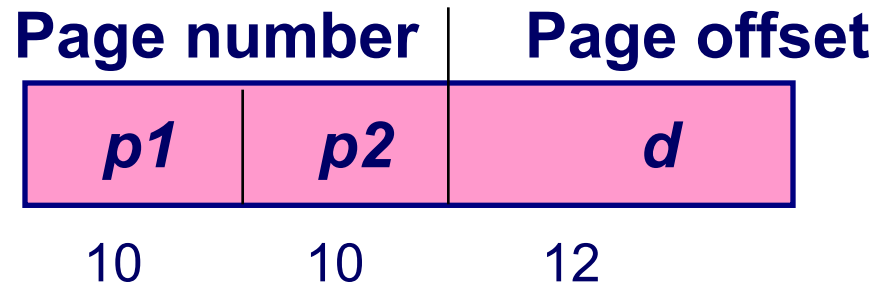
Two-Level Page-Table Scheme

(Fig. 9.15)



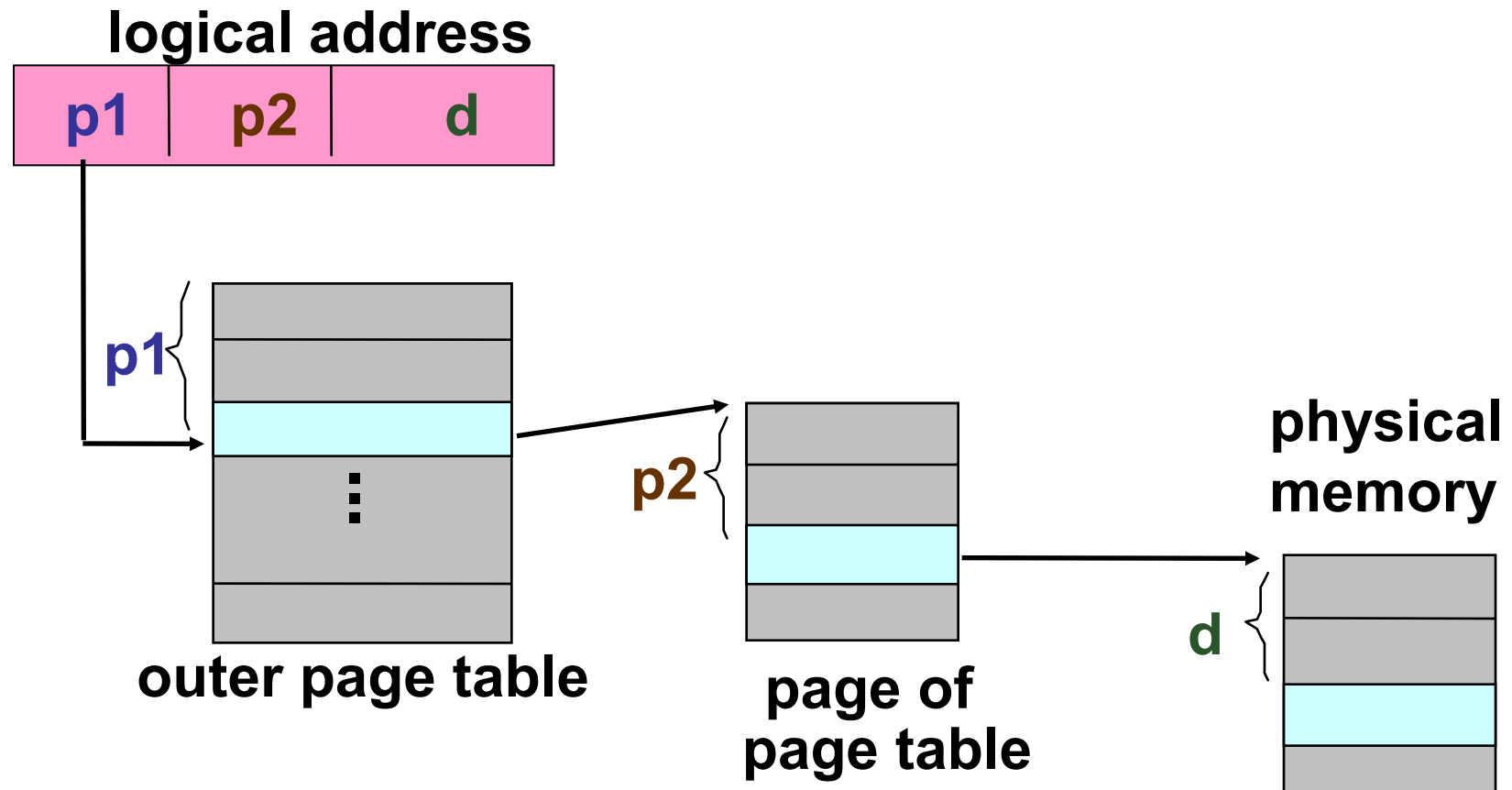
Two-Level Paging Example

- Logical address (e.g., on 32-bit machine with 4KB page size)



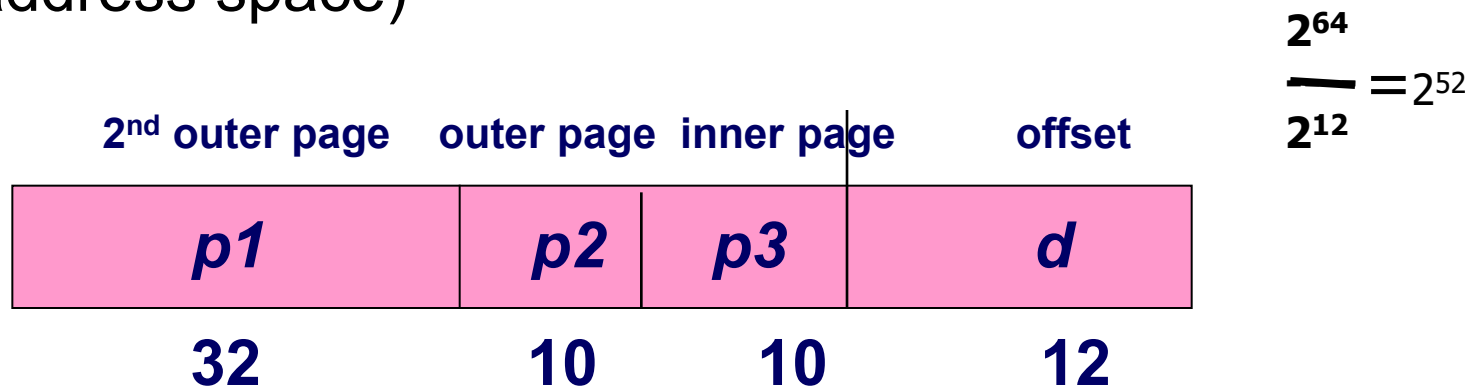
- P_1 : an index into the outer page table
 - p_2 : the displacement within the page of the inner page table.
-
- Used in Pentium II (page table entry=4 bytes)

Address Translation for a Two-Level 32-bit Paging Architecture (Fig. 9.16)



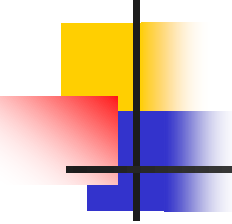
Hierarchical Paging (Cont.)

- For a system with a large logical address space, more level paging scheme is needed.
- Example: 3-level paging scheme (64-bit logical-address space)



- Since each level is stored as a separate table in memory, converting a logical address to a physical one may take more times of memory accesses.

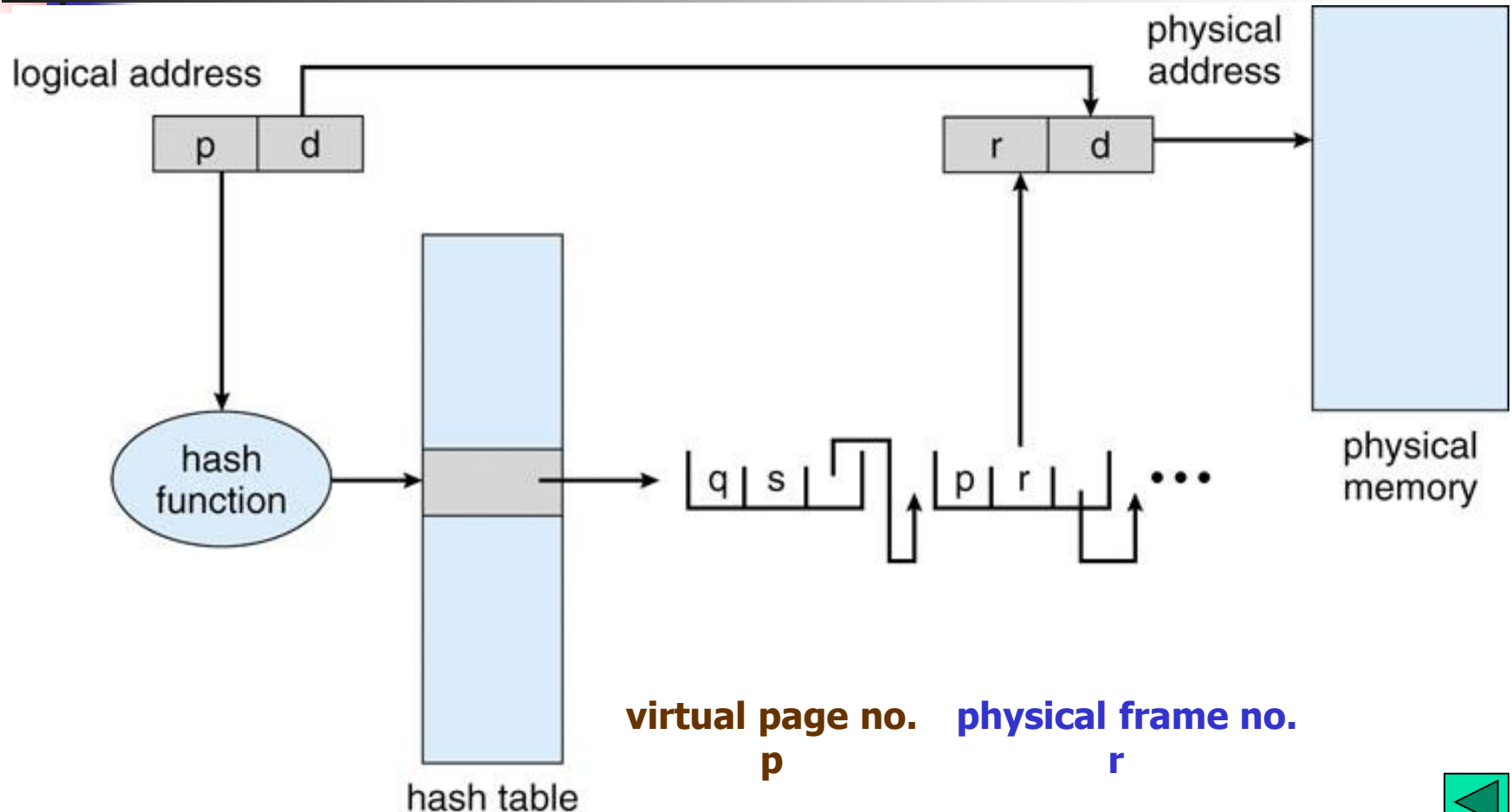


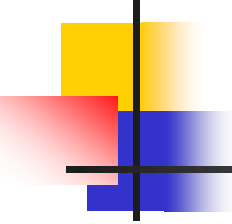


9.4.2 Hashed Page Tables

- Common used in handling address spaces > 32 bits.
- The virtual page number is hashed into a page table. This page table contains a chain of elements hashing to the same location.
- Virtual page numbers are compared in this chain searching for a match. If a match is found, the corresponding physical frame is extracted.

Hashed Page Table (Fig. 9.17)





9.4.3 Inverted Page Tables

- Usually, each process has a page table. However, each table may consist of millions entries.
 - This may consumes large amounts of physical memory.
- To solve this, the *inverted page table* can be used.
 - Only one inverted page table in the system
 - Only one entry for each real page (or frame) of physical memory
 - Entry consists of the virtual address of the page stored in that real memory location, with information about the process that owns that page.
 - e.g., each entry contains (process id, virtual address)

Inverted Page table

0	P	0
1	C	1
2	P	1
3		
4	P	3
5	C	0
6	P	2

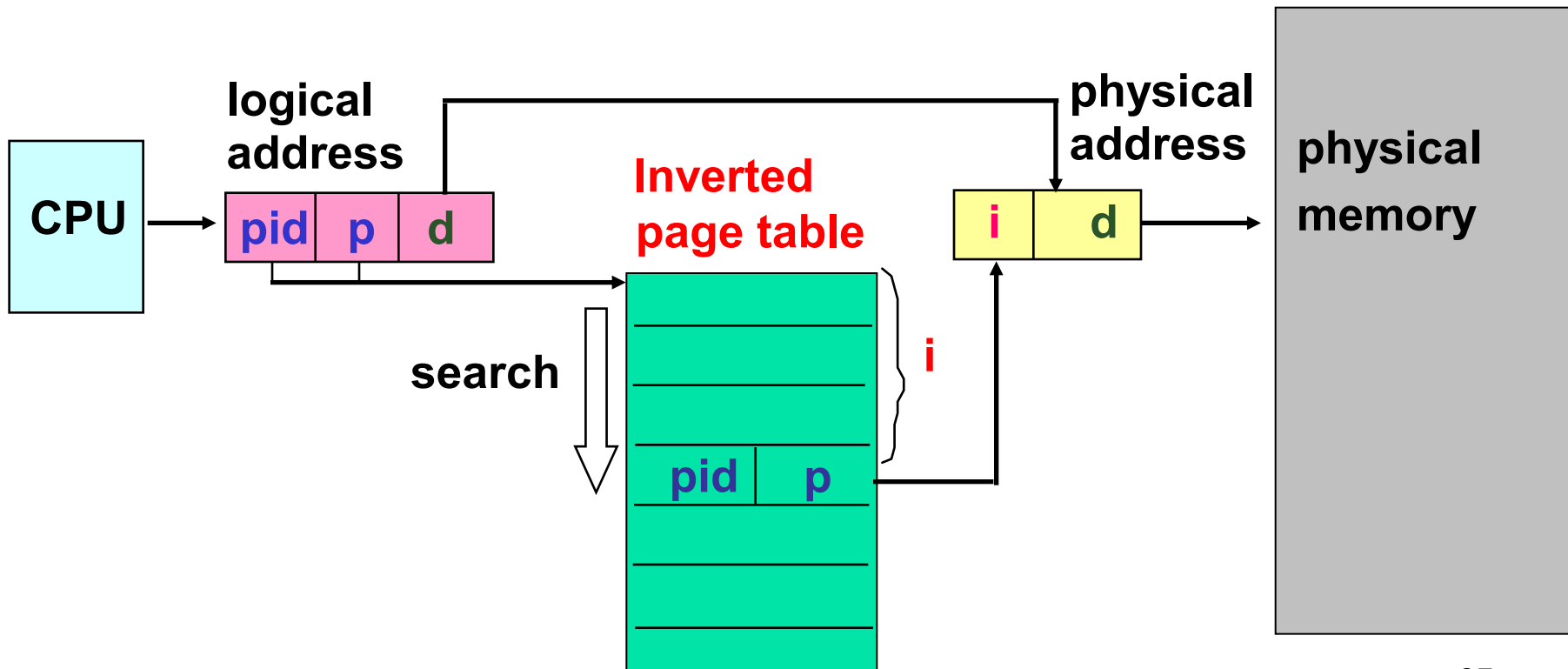
0	5
1	1

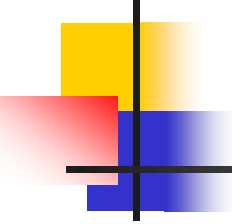
page table

0	0
1	2
2	6
3	4

Address Translation in a Inverted Page Table (Fig. 9.18)

- virtual address: $\langle \text{process-id, page-number, offset} \rangle$
- inverted page-table entry: $\langle \text{process-id, page-number} \rangle$

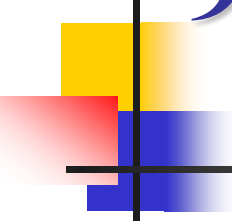




Inverted Page Table (Cont.)

- e.g., 64-bit UltraSPARC, PowerPC, IBM RT
- Decreases memory needed to store each page table
- Increases time needed to search the table when a page reference occurs.
- Use a hash table to limit the search to one — or at most a few — page-table entries.
- To improve performance, TLB is used to hold recently located entries.





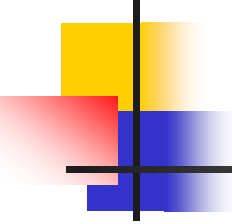
9.4.4 Oracle SPARC Solaris

- **Consider modern, 64-bit OS example with tightly integrated HW**
 - **Goals are efficiency, low overhead**
- **Based on hashing, but more complex**
- **Two hash tables**
 - **One kernel and one for all user processes**
 - **Each maps memory addresses from virtual to physical memory**
 - **Each entry represents a contiguous area of mapped virtual memory,**
 - **More efficient than having a separate hash-table entry for each page**
 - **Each entry has base address and span (indicating the number of pages the entry represents)**

Oracle SPARC Solaris (Cont.)

- **TLB holds translation table entries (TTEs) for fast hardware lookups**
 - A cache of TTEs reside in **a translation storage buffer (TSB)**
 - Includes an entry per recently accessed page
- **Virtual address reference causes TLB search**
 - If miss, hardware walks the in-memory TSB looking for the **TTE corresponding to the address**
 - If match found, the CPU copies the TSB entry into the TLB and translation completes
 - If no match found, kernel interrupted to search the hash table
 - The kernel then creates a TTE from the appropriate hash table and stores it in the TSB, Interrupt handler returns control to the MMU, which completes the address translation.



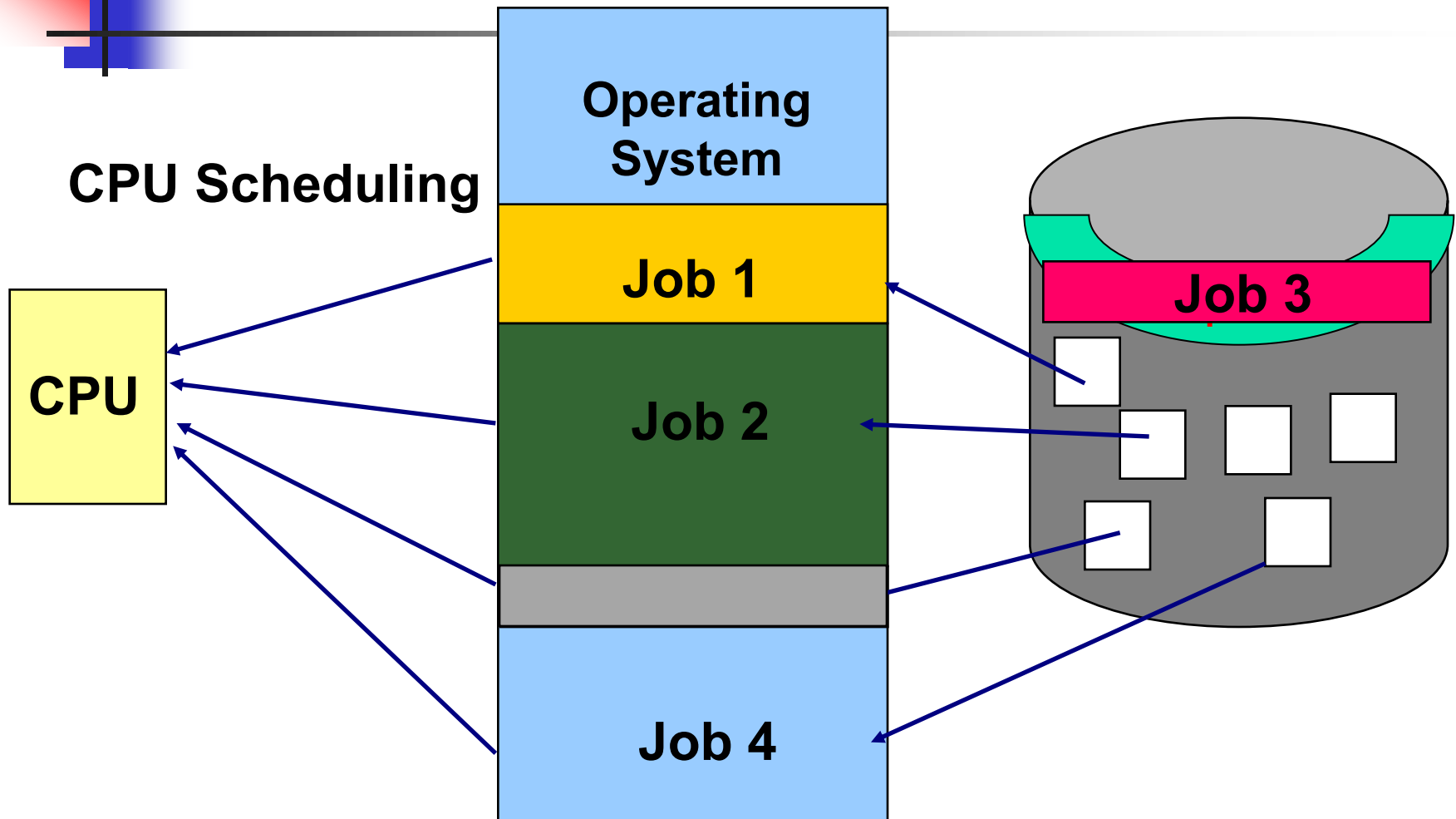


9.5 Swapping

- A process can be **swapped** temporarily out of memory to a **backing store**, and then brought back into memory for continued execution.
 - Used to free large amounts of memory
- **Backing store**
 - fast disk large enough to accommodate copies of all memory images for all users and it must provide direct access to these memory images

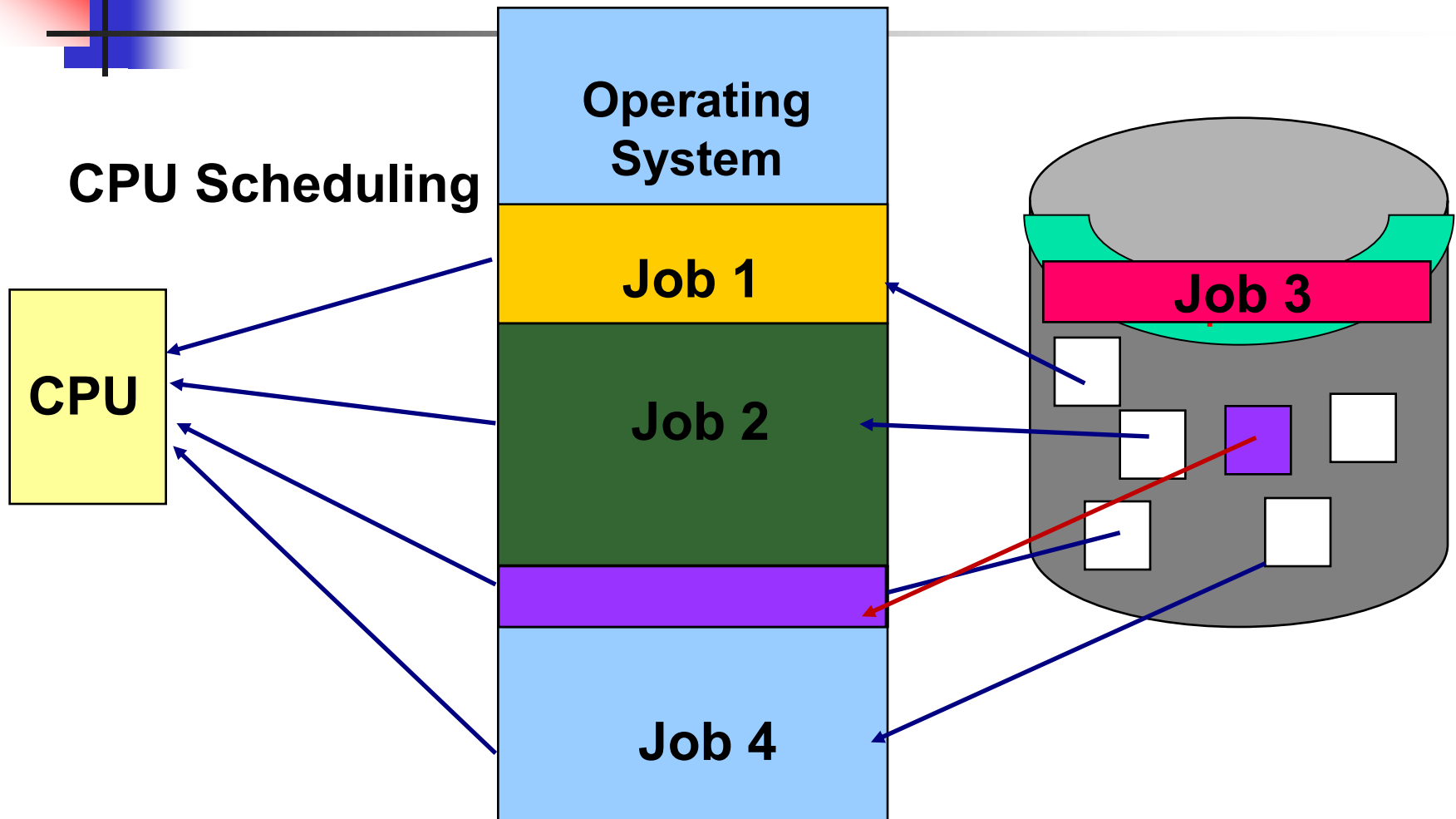
Swapping

When free memory is not enough!

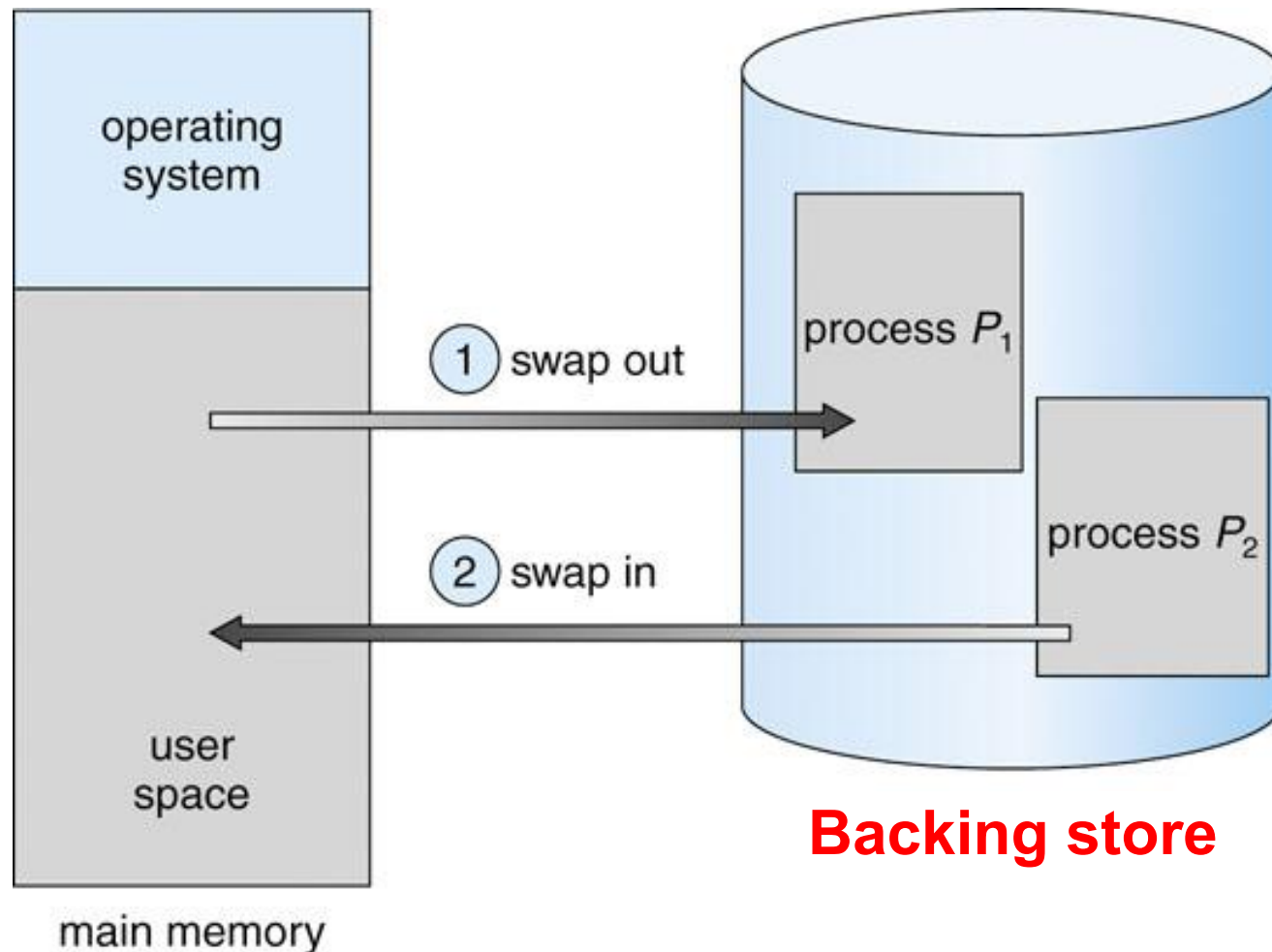


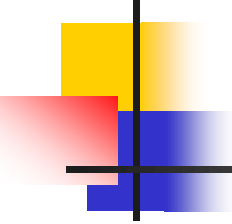
Swapping

When a higher-priority process is ready to run!



Swapping of two processes using a disk as a backing store (Fig. 9.19)

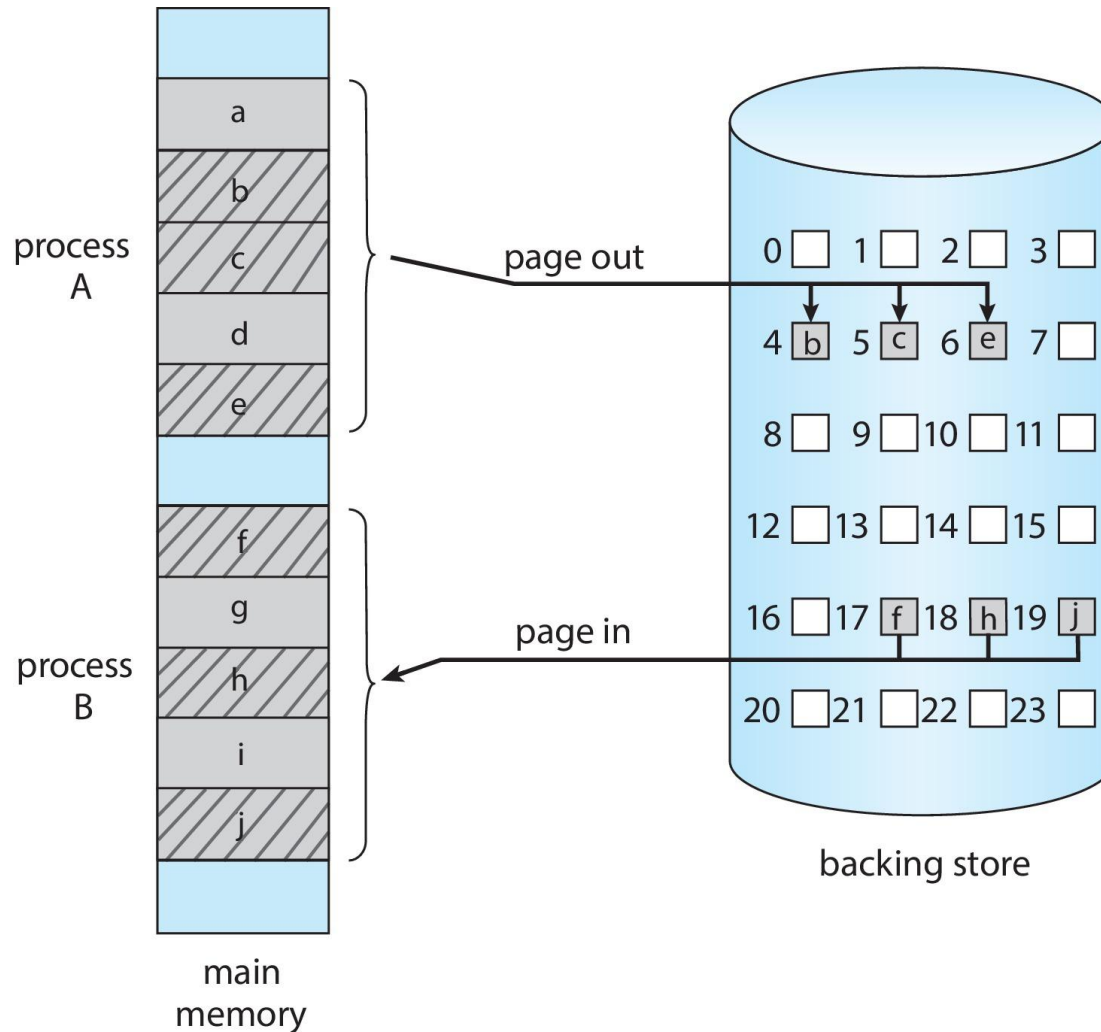




Swapping (Cont.)

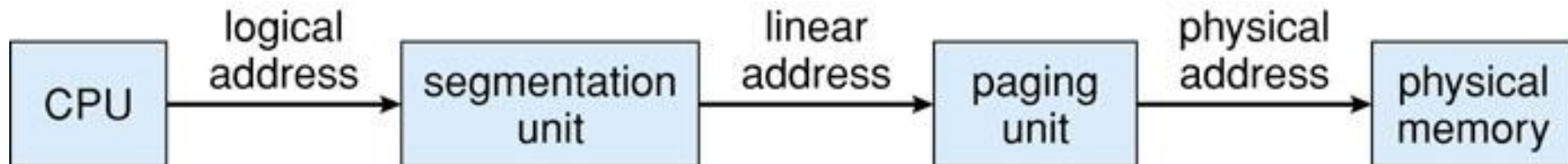
- Major part of swap time is transfer time; total transfer time is directly proportional to the amount of memory swapped.
- Most systems, including Linux, and Windows, now use a variation of swapping in which pages of a process – rather than an entire process – can be swapped.
 - Swapping with paging
 - Page in
 - Page out

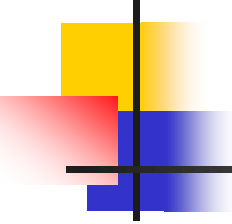
Swapping with Paging (Fig. 9.20)



9.6 Example: The Intel 32 and 64-bit Architecture

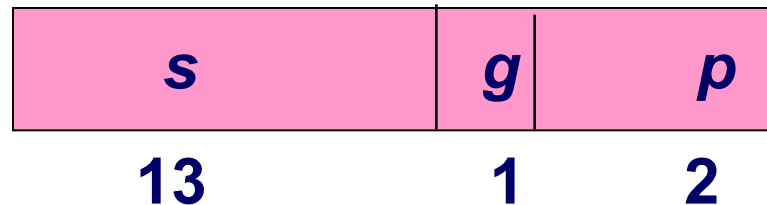
- Both paging and segmentation have advantages and disadvantages.
- Intel Pentium architecture supports both pure segmentation and segmentation with paging.
- Logical to physical address translation in IA-32.





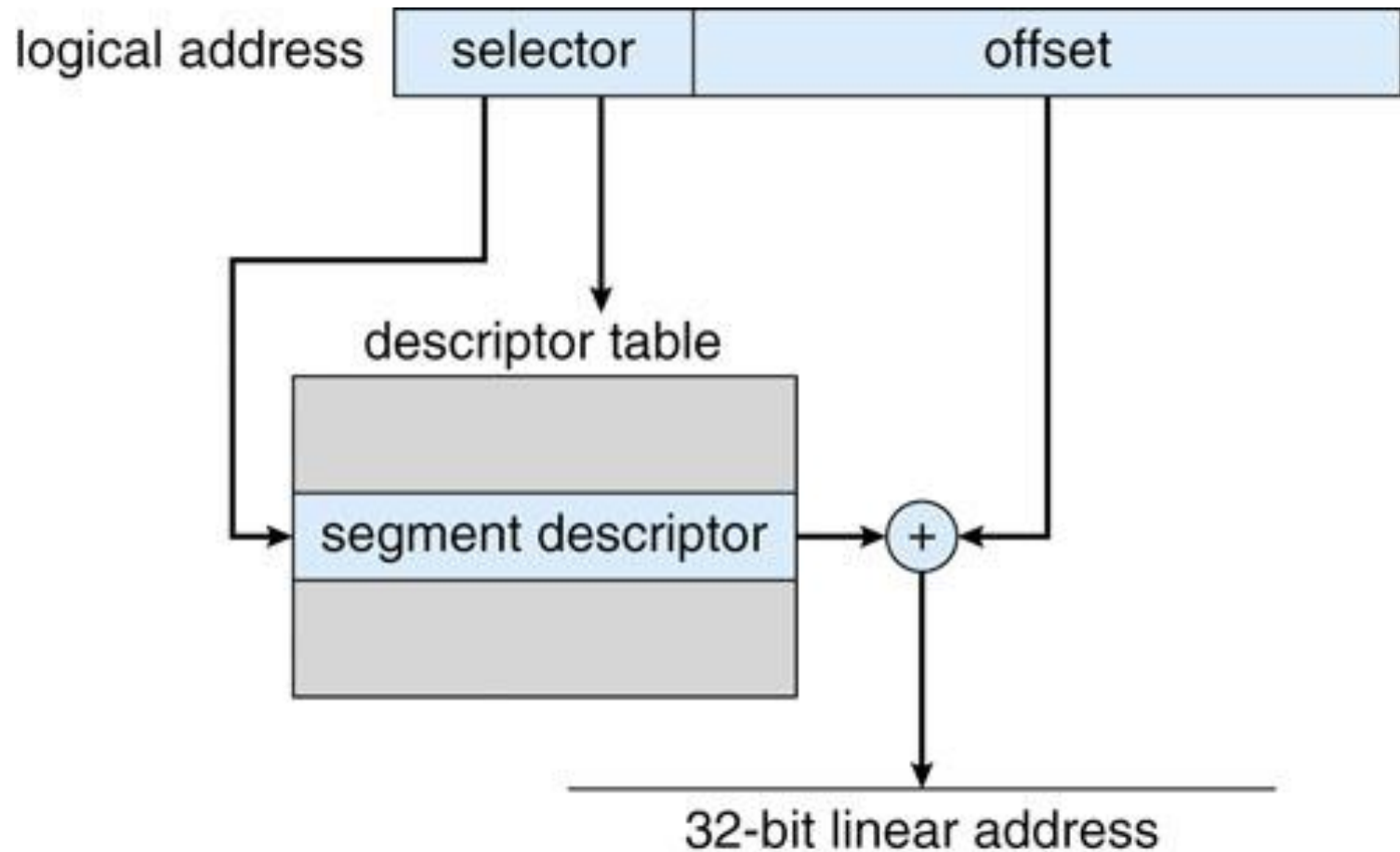
9.6.1.1 IA-32 Segmentation

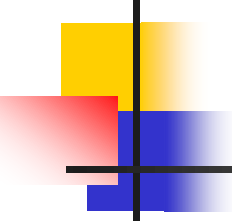
- **Logical address is a pair (selector, offset)**
 - **Selector: 16-bit**
 - Contains 13-bit segment number, 1-bit descriptor table indicator, 2-bit protection



- **Offset: 32-bit**
 - Offset within the segment

IA-32 Segmentation



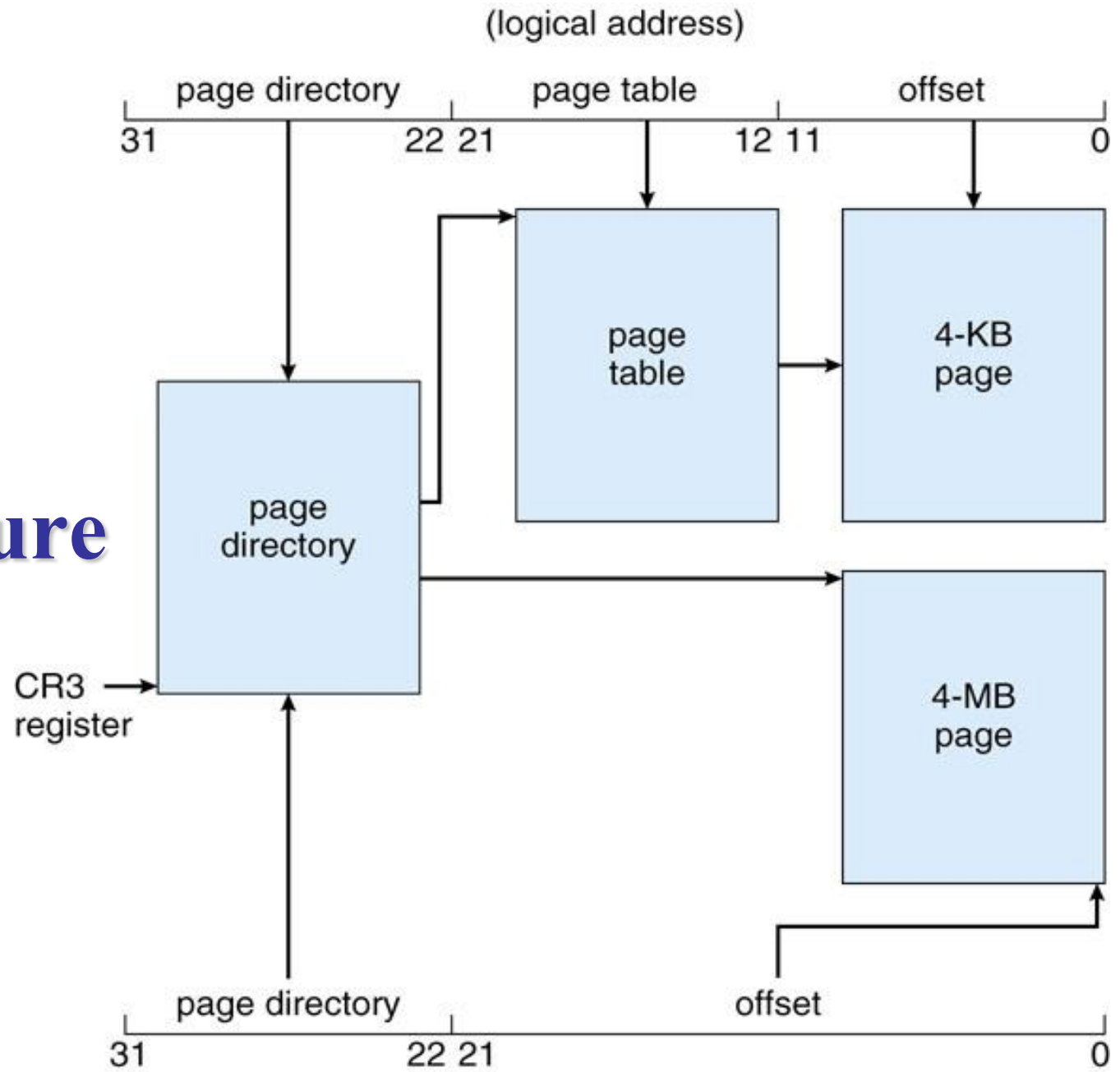


9.6.1.2 IA-32 Paging

- uses **segmentation with paging** for memory management with a **two-level paging** scheme.
- 32-bit linear address: (4-KB pages)

Page number		Page offset
<i>p1</i>	<i>p2</i>	<i>d</i>
10	10	12

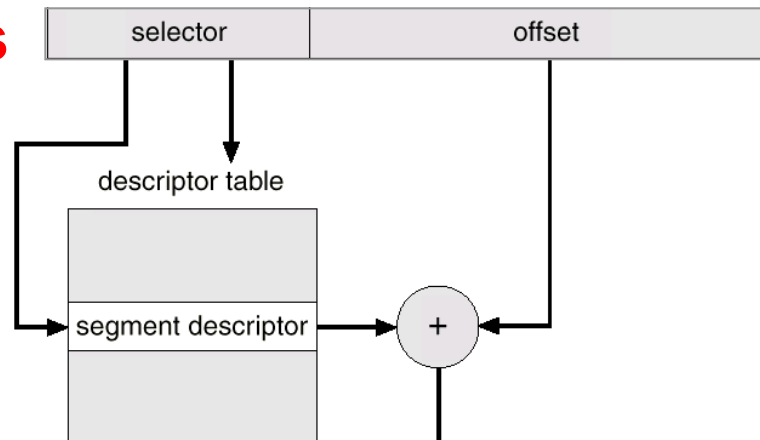
Paging in IA-32 Architecture



IA-32 address translation

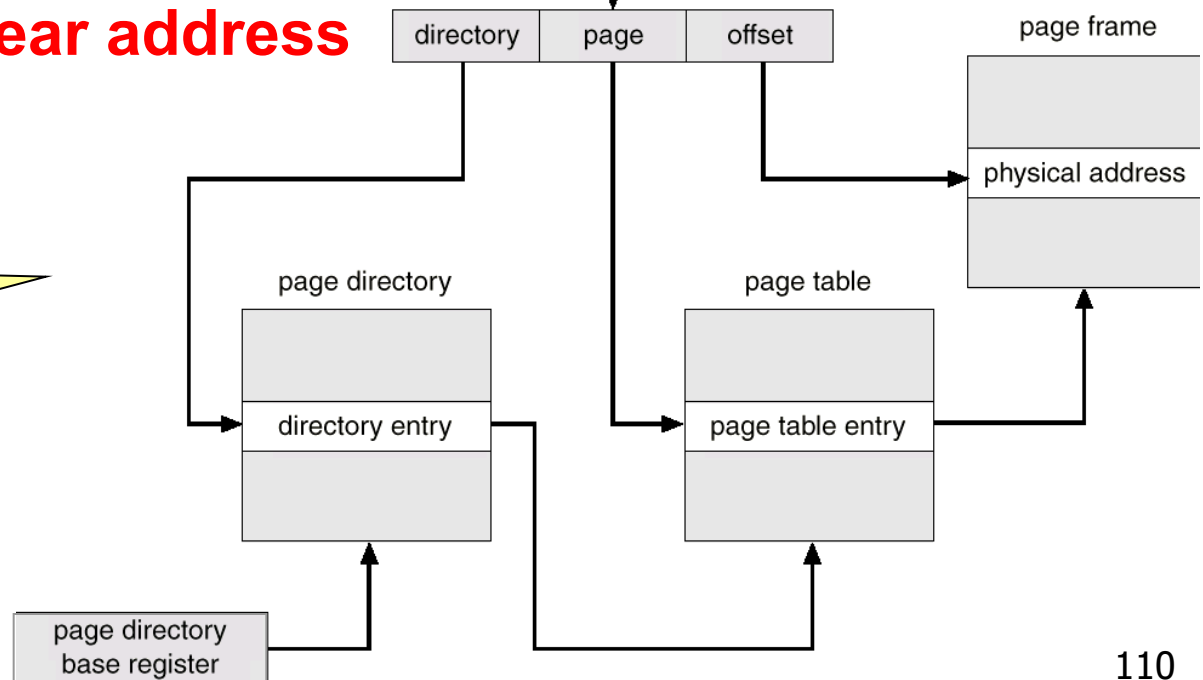
logical address

Segment
translation

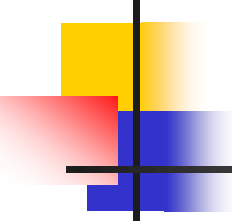


Linear address

Two-level
paging

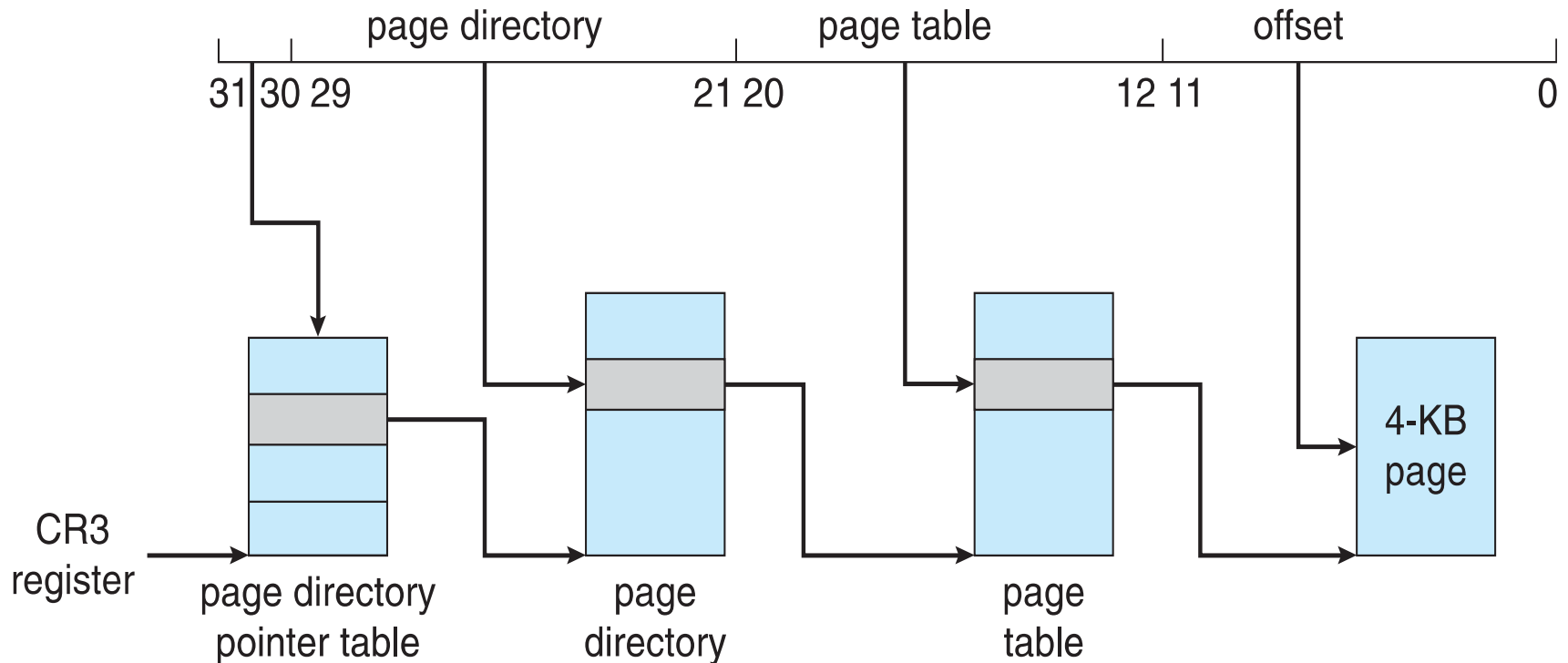


Intel IA-32 Page Address Extensions



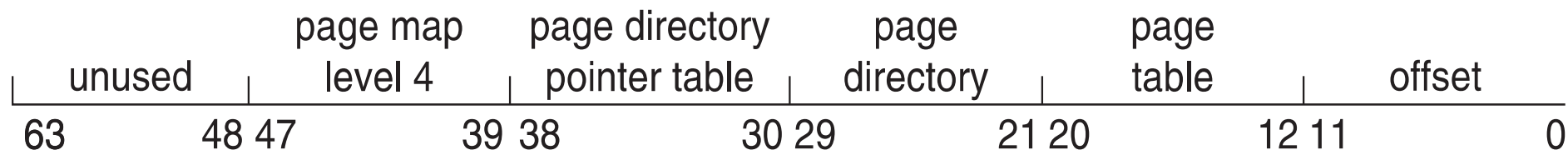
- 32-bit address limits led Intel to create **page address extension (PAE)**, allowing 32-bit apps access to more than 4GB of memory space
 - Paging went to a 3-level scheme
 - Top two bits refer to a **page directory pointer table**
 - Page-directory and page-table entries moved to 64-bits in size
 - Net effect is increasing address space to 36 bits – 64GB of physical memory

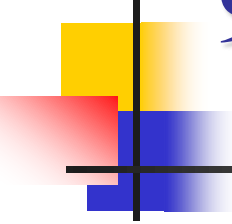
Intel IA-32 Page Address Extensions



9.6.2 x86-64

- Current generation Intel x86 architecture
- 64 bits is ginormous (> 16 exabytes)
- In practice only implement 48 bit addressing
 - Page sizes of 4 KB, 2 MB, 1 GB
 - Four levels of paging hierarchy
- Can also use PAE so virtual addresses are 48 bits and physical addresses are 52 bits





9.7 Example: ARMv8 Architecture

- Dominant mobile platform chip (Apple iOS and Google Android devices for example)
- Modern, energy efficient, 64-bit ARMv8
- 4 KB, 16KB, and 64 KB pages
- 2 MB, 1 GB, 32MB, and 512MB pages (termed **regions**)
- For 4 KB and 16KB pages, up to four-level paging may be used.
- For 64 KB and 16KB pages, up to three-level paging may be used.

ARM four-level hierarchical paging

